

DD2380 Artificial Intelligence

Search

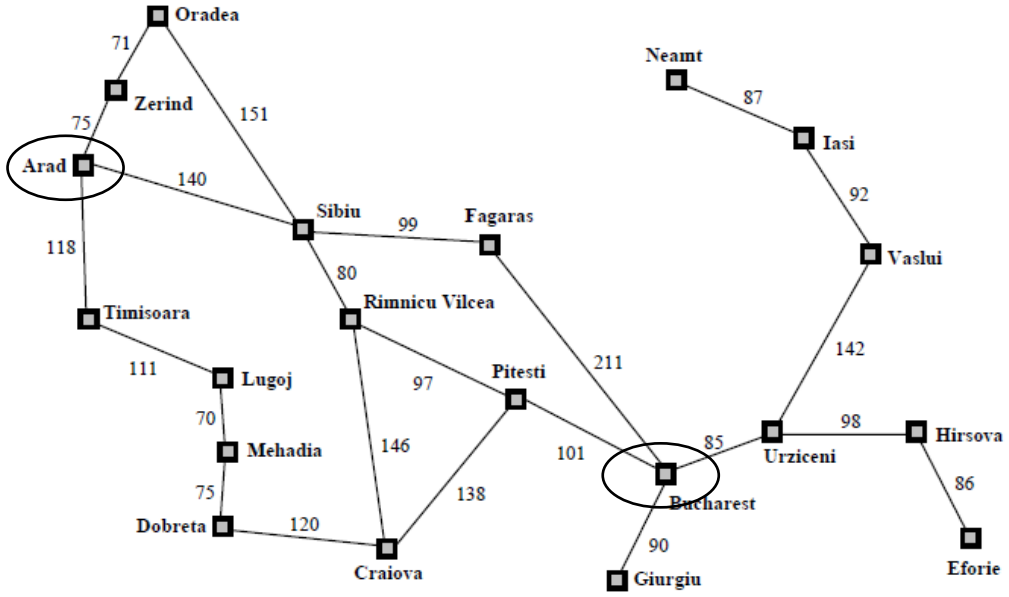
Lectures

- Introduction and Practicalities
- AI and society
- **Problem solving**
  - Search
  - Games
- Logic and planning
- Taming uncertainty
  - Probabilistic reasoning, Bayesian Networks
  - Hidden Markov Models
- Decision making under uncertainty
  - Decision making using MDPs and POMDPs
  - Reinforcement learning
- Multi agent systems + robotics
- Machine learning
- Natural language processing and conversational systems
- Computer vision
- Generative AI and wrap-up

Problem solving via search

- A problem is composed of 4 components:
  - An initial state
  - Successor function, new_state = f(old_state, action)
  - A goal test
  - A path cost specification
- State space is the set of all states *reachable* from the initial state
- A solution is a sequence of actions.
- An optimal solution is a solution with minimum cost.

Example: Routing problem



Initial state:

Arad

Successor function:

Zerind = f(Arad,…)

…

Goal test:

Is the state Bucharest?

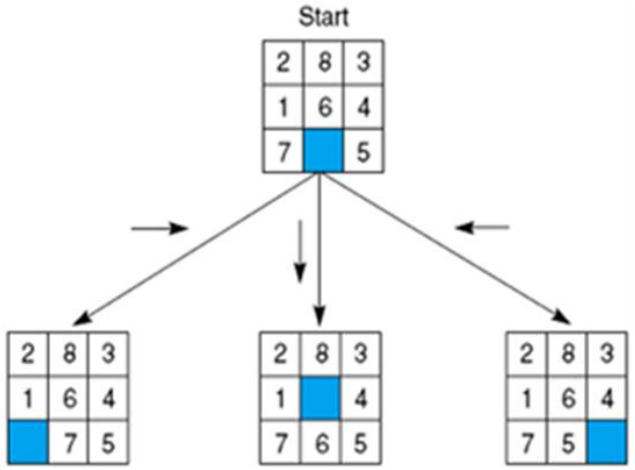
Path cost specification: distance traveled

Example: 8-puzzle

Initial state:



Successor function:

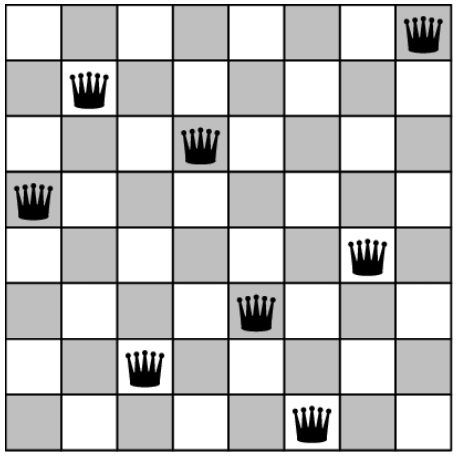


Goal test:



Path cost specification: number of moves

Example: 8-queens



Initial state:

?

Successor function:

?

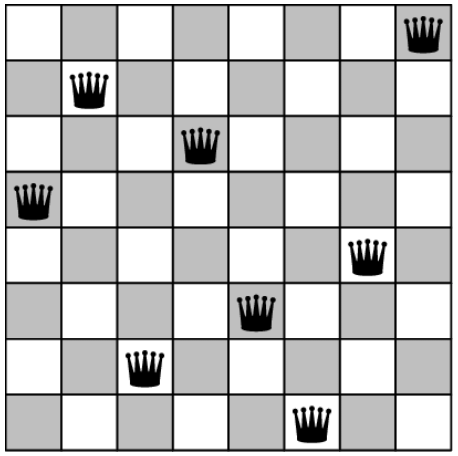
Goal test:

?

Path cost specification:

?

Example: 8-queens



Initial state:

An empty chessboard

Successor function:

Add one queen, ideally one column at a time

Goal test:

No queen is under attack, and all are on the board

Path cost specification:
Number of moves (1 per placement or movement)

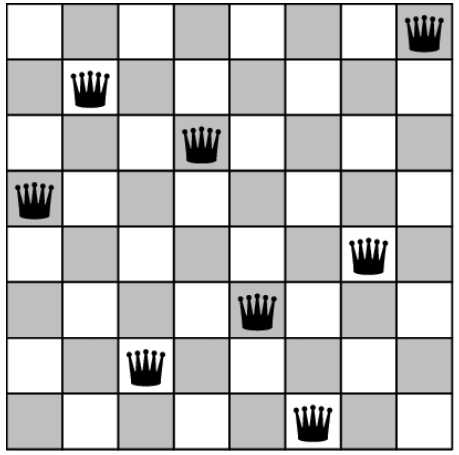
Characterization of search methods

- Time and space complexity is measured in terms of
  - **b** branch factor (max num of successors of any node)
  - **d** depth of optimal solution
  - **m** maximum length of a path in the state space (could be ∞)

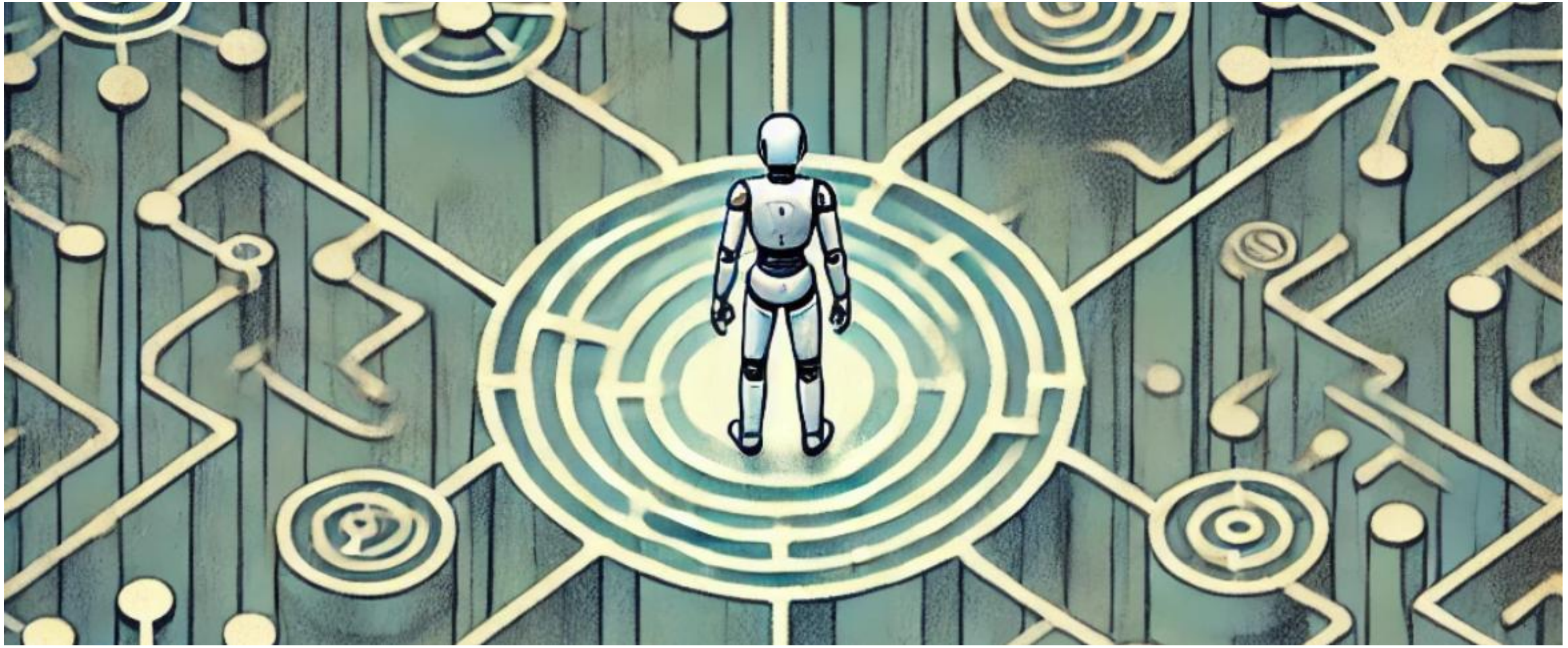
What is **b**?



What is **d**? and **m**?



Uninformed search



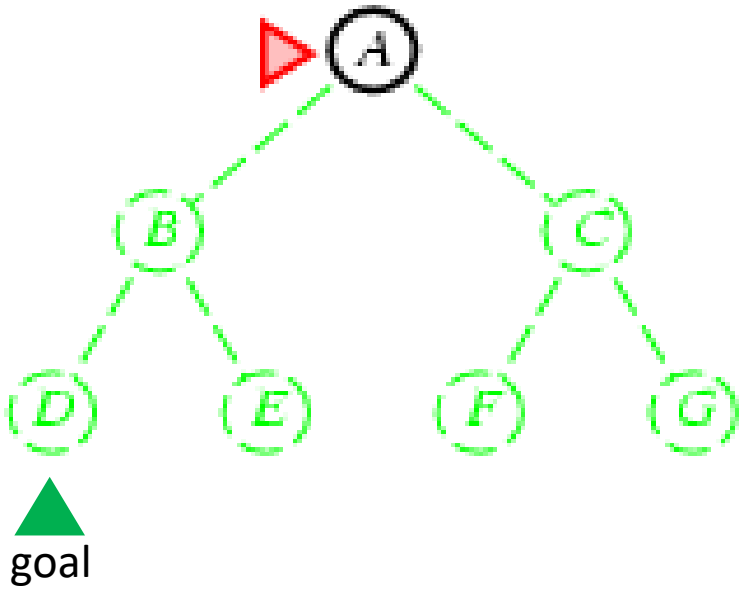
Generic tree search algorithm

- Add root node (start state) to container
- Repeat
  - If container empty → FAILED
  - Choose a node to take out from the container
  - If node corresponds to goal → FOUND IT!
  - Add successors / children to the container

- Search method depends on how we add/take out node from the container

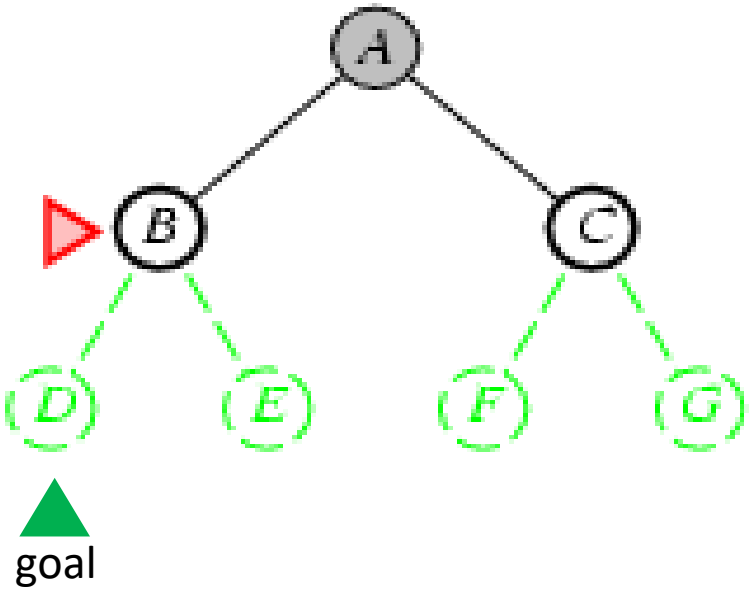
Breadth-first search (BFS)

- Expand the shallowest unexpanded node
- Implementation:
  - *Fringe is* a FIFO queue, i.e., new successors go at the end



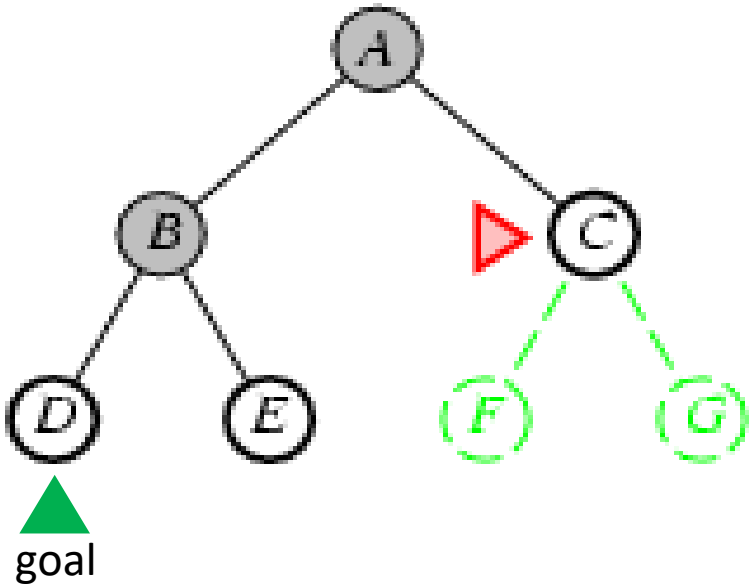
Breadth-first search (BFS)

- Expand the shallowest unexpanded node
- Implementation:
  - *Fringe is* a FIFO queue, i.e., new successors go at the end



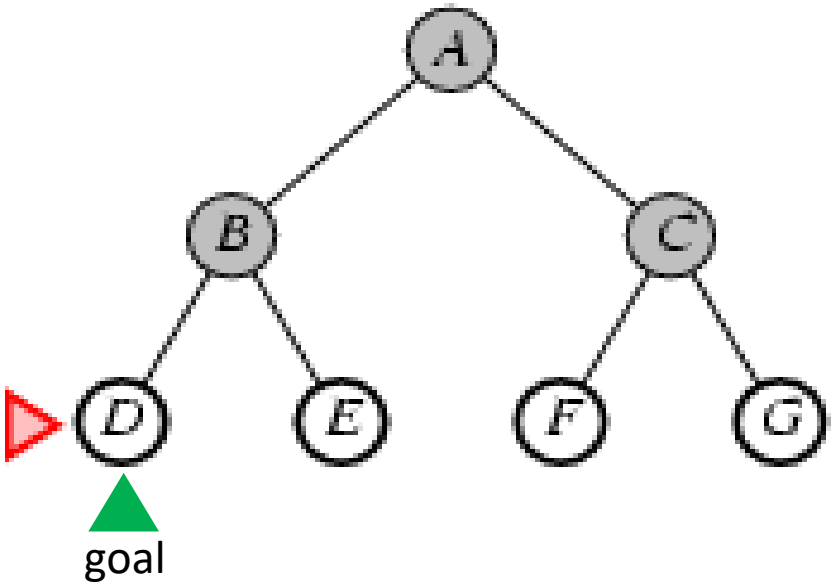
Breadth-first search (BFS)

- Expand the shallowest unexpanded node
- Implementation:
  - *Fringe is* a FIFO queue, i.e., new successors go at the end



Breadth-first search (BFS)

- Expand the shallowest unexpanded node
- Implementation:
  - *Fringe is* a FIFO queue, i.e., new successors go at the end

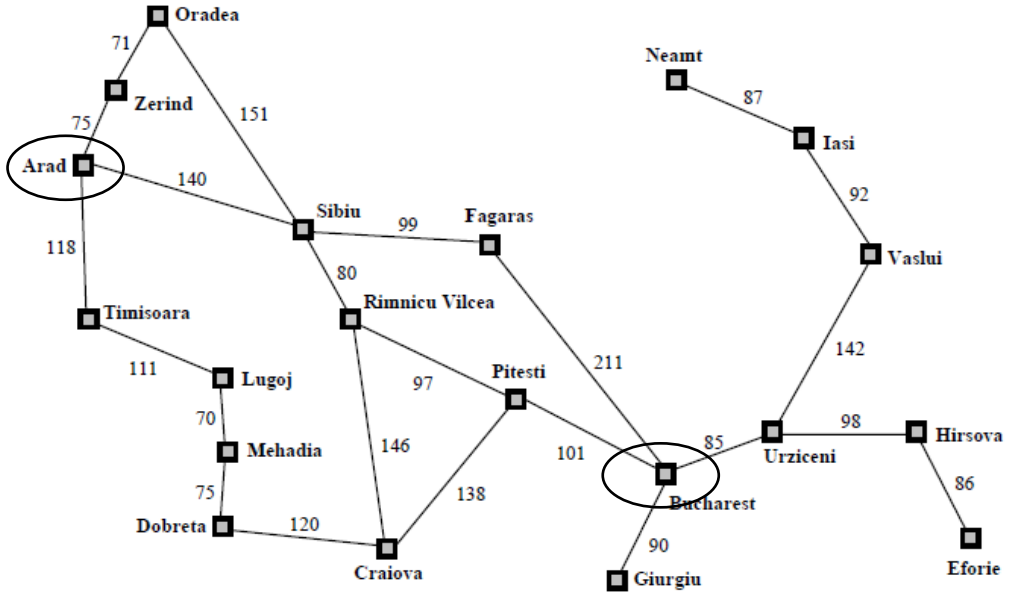


Breadth-first search properties
(Tree version with goal test at node generation)

- Completeness: Yes
- Time: $1 + b + b^2 + b^3 + ... + b^d = O(b^d)$
- Space: keeps all nodes in memory $O(b^d)$
  - This can easily become a bottleneck
- Optimal: Not in general

  - **b** branch factor
  - **d** depth of optimal solution

Example: Routing problem (is bfs optimal here?)



Initial state:

Arad

Successor function:

Zerind = f(Arad,…)

…

Goal test:

Is the state Bucharest?

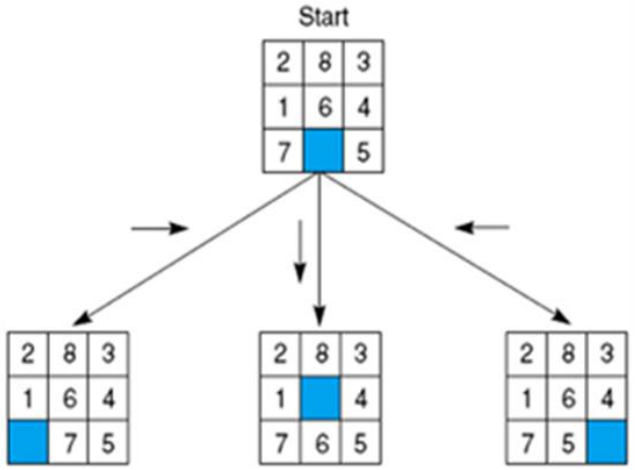
Path cost specification: distance traveled

Example: 8-puzzle (is bfs optimal here?)

Initial state:



Successor function:



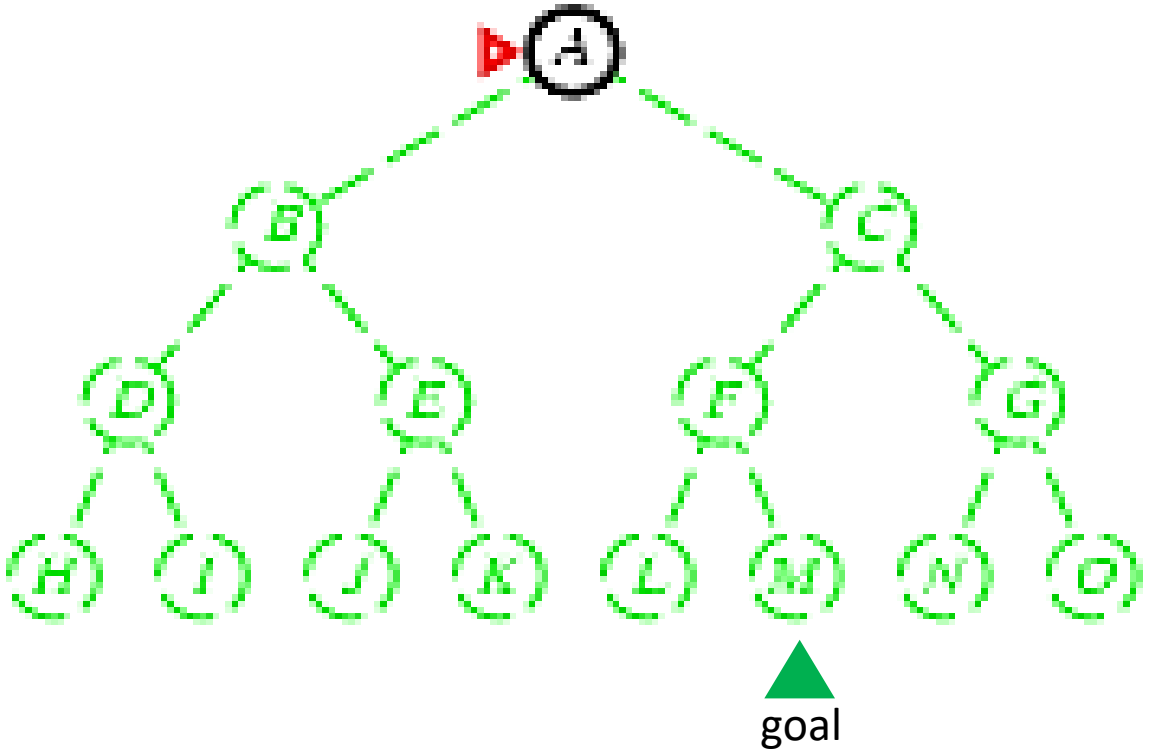
Goal test:



Path cost specification: number of moves

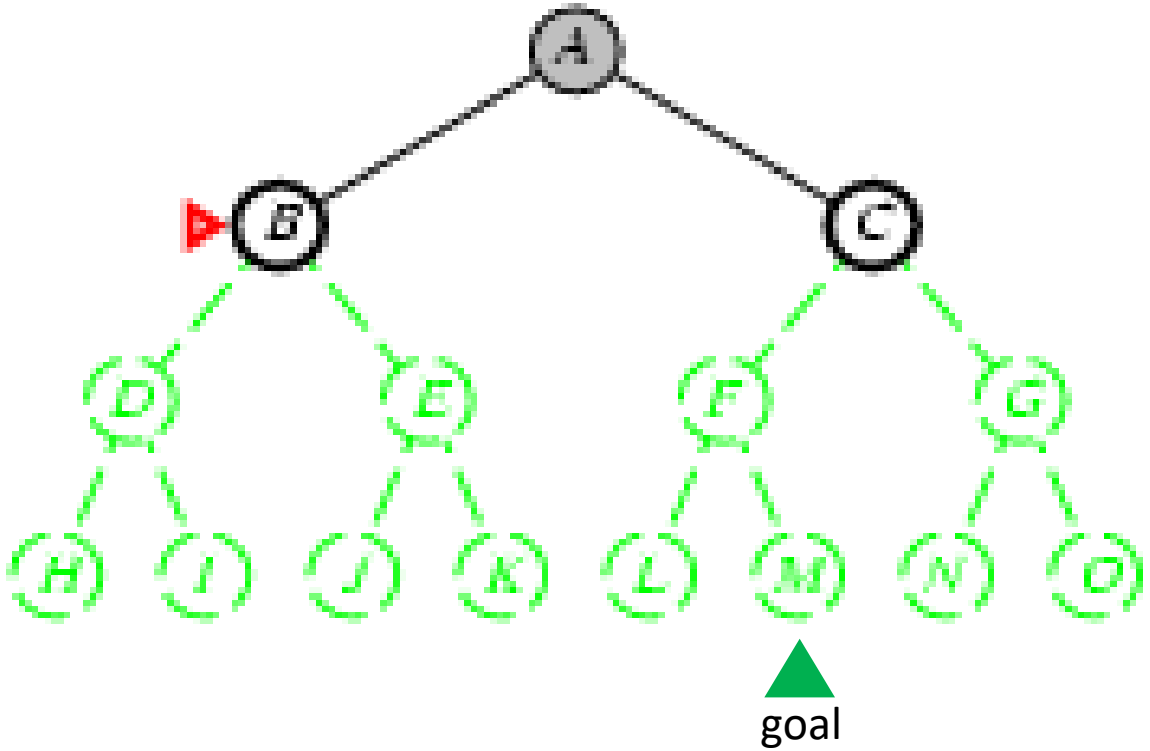
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



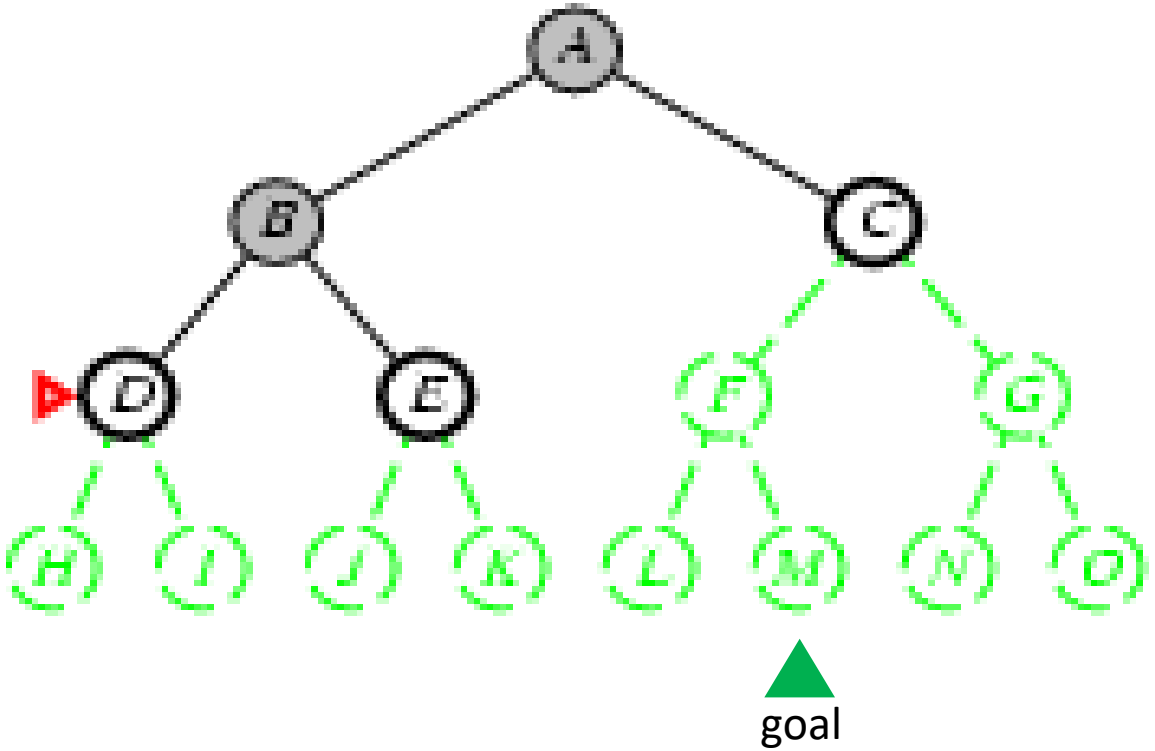
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



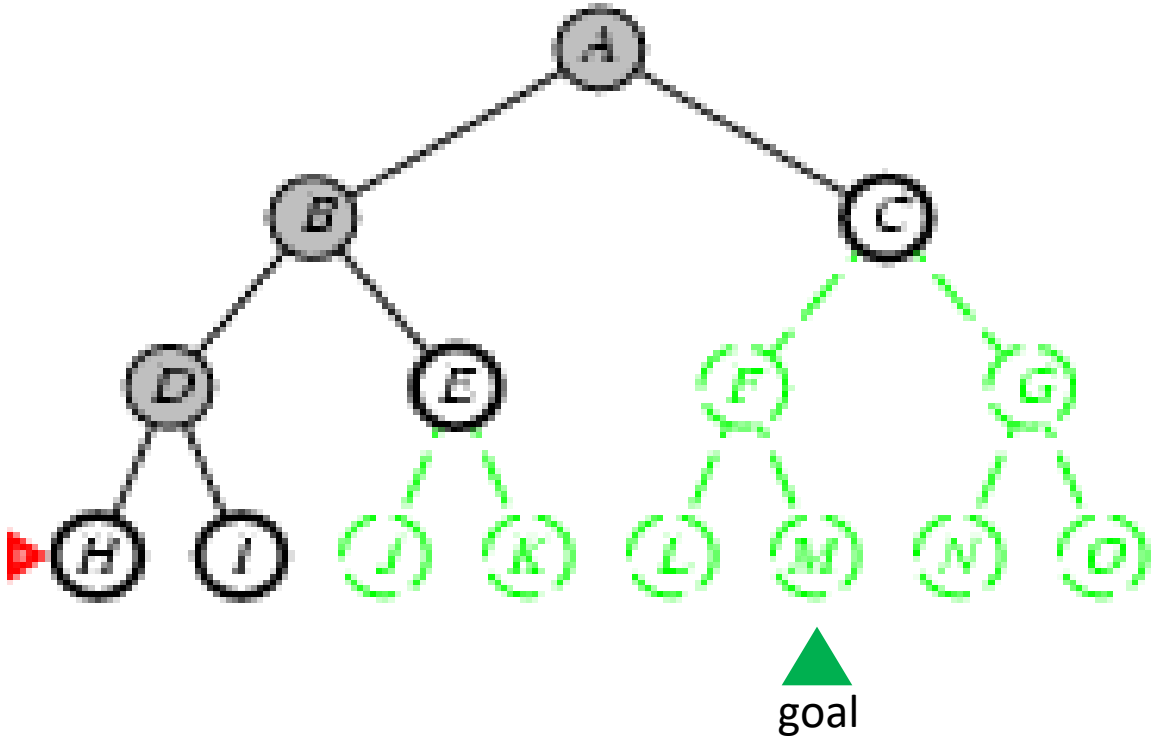
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



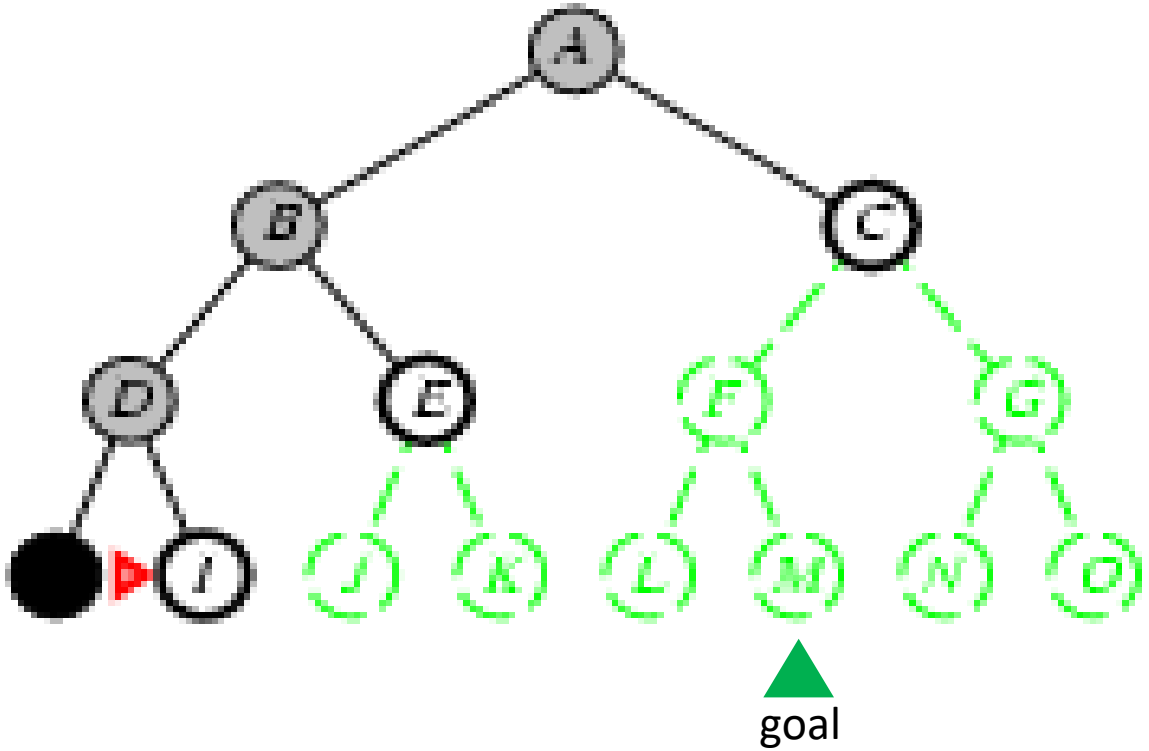
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



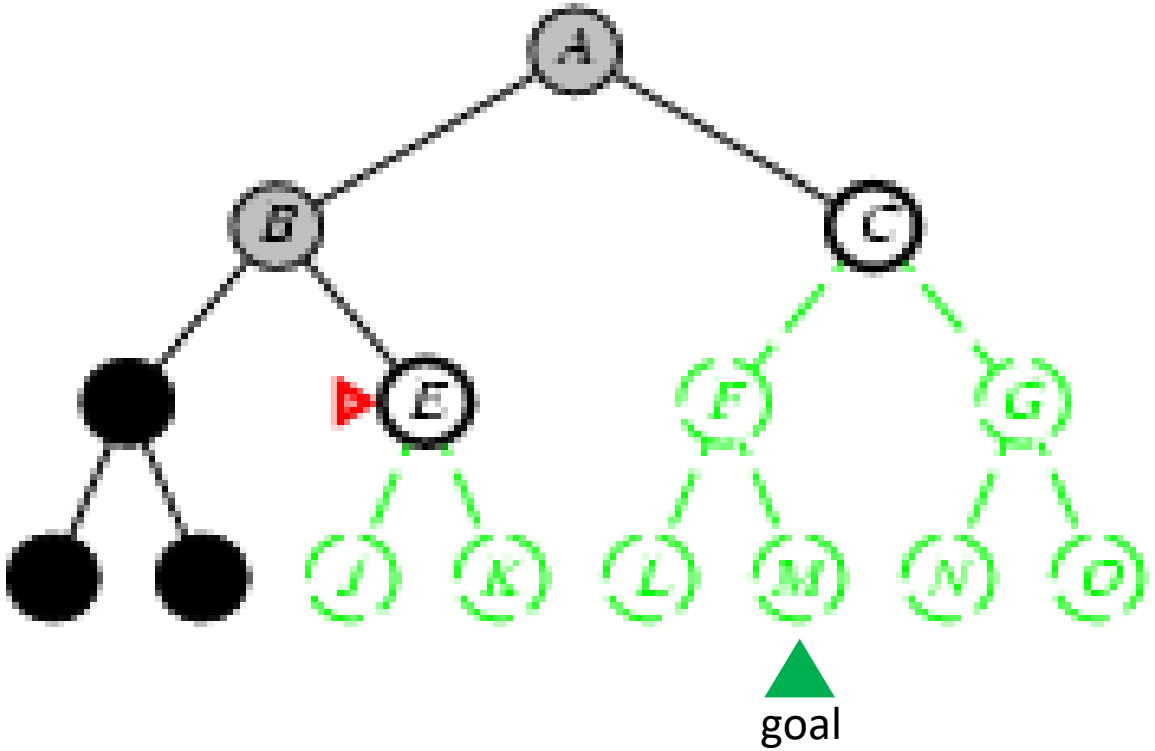
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



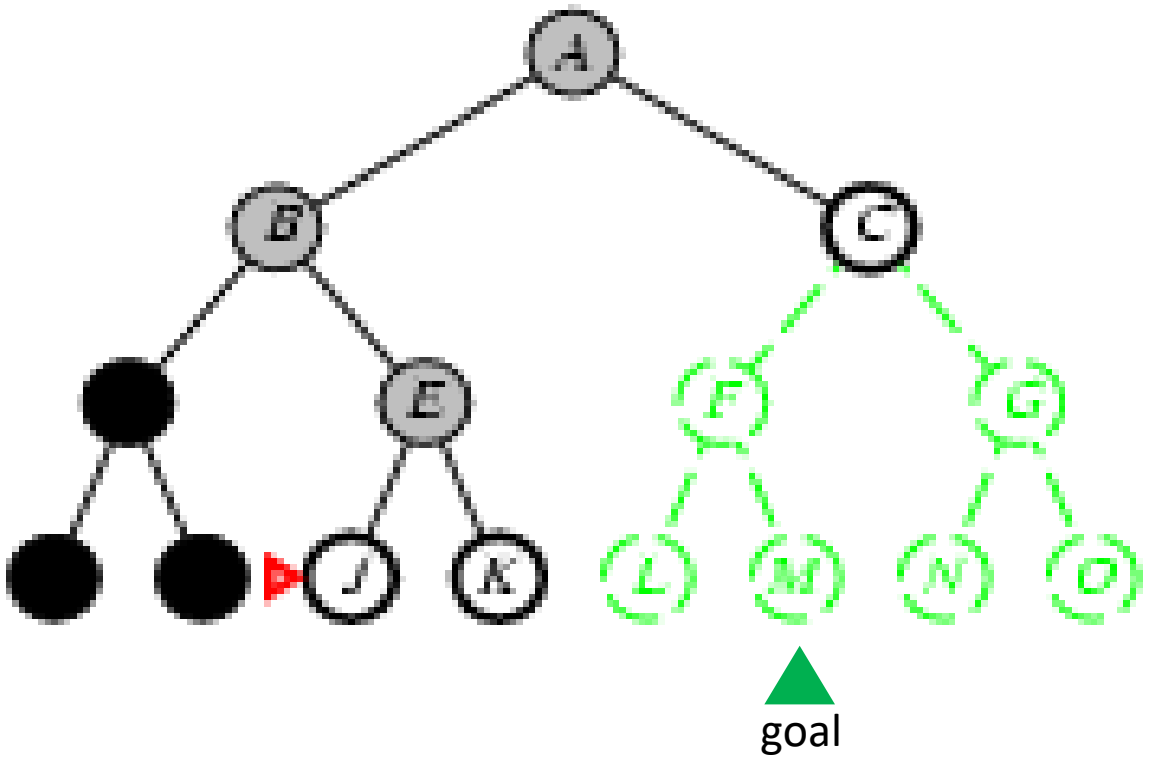
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



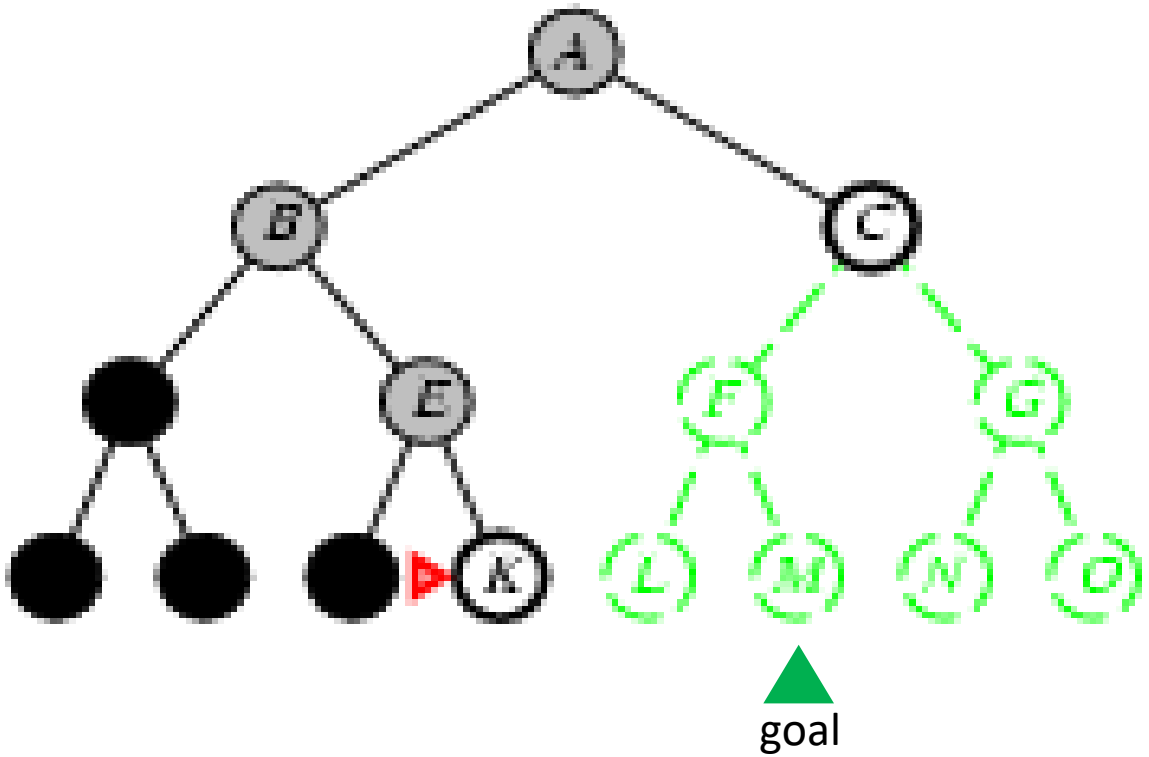
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



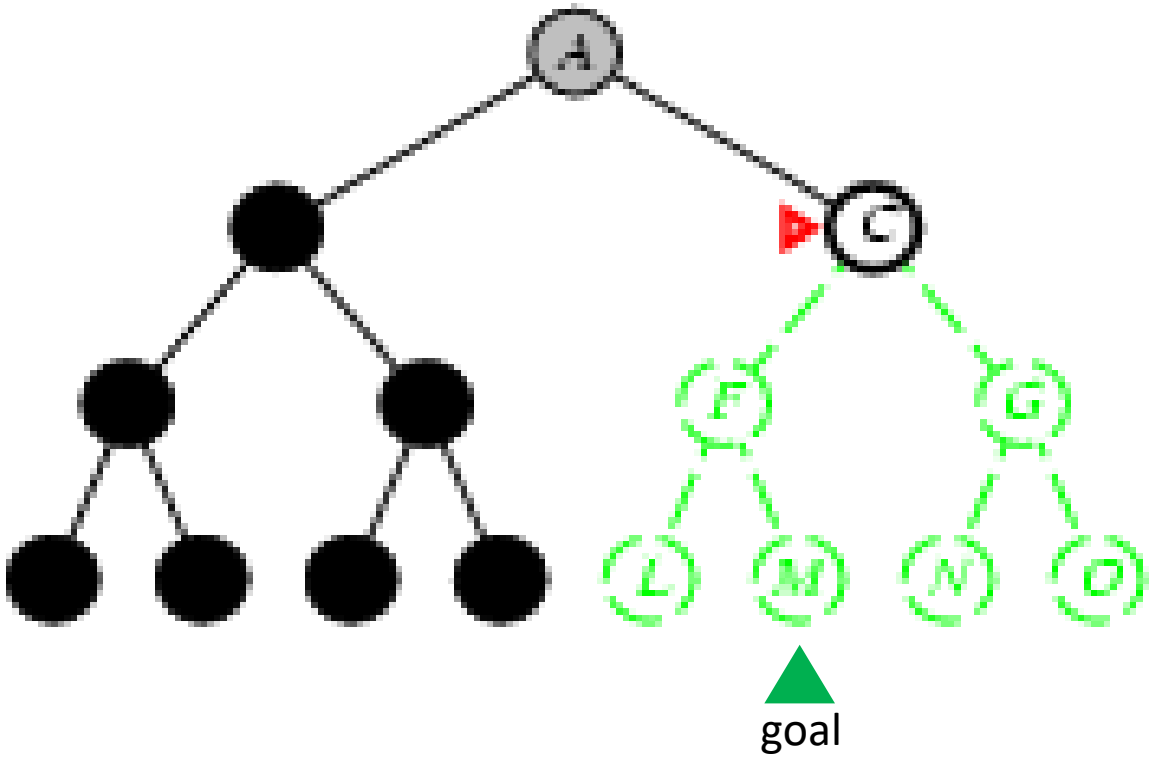
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



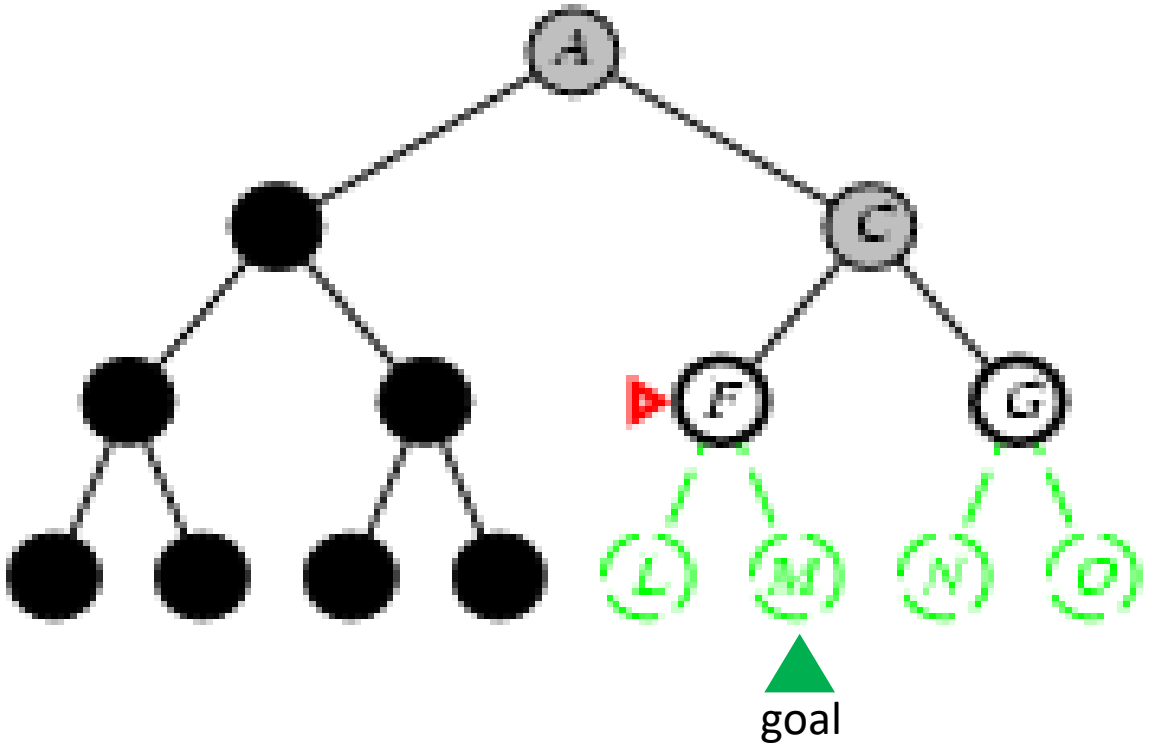
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



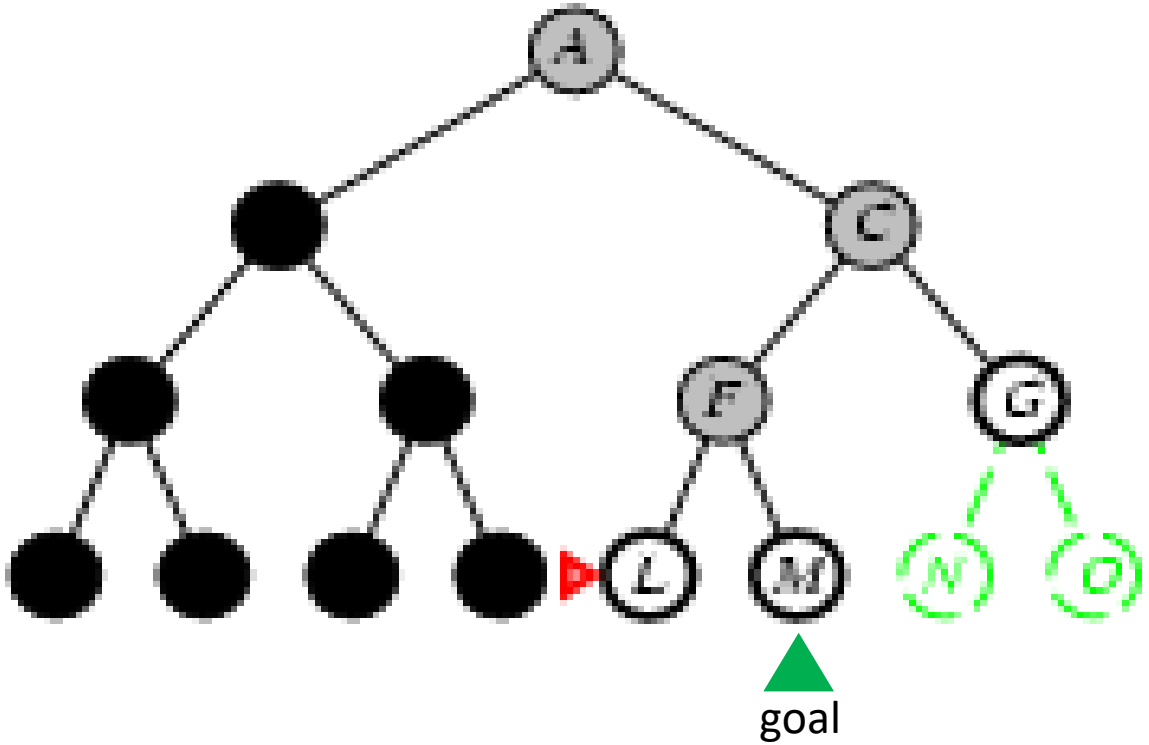
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



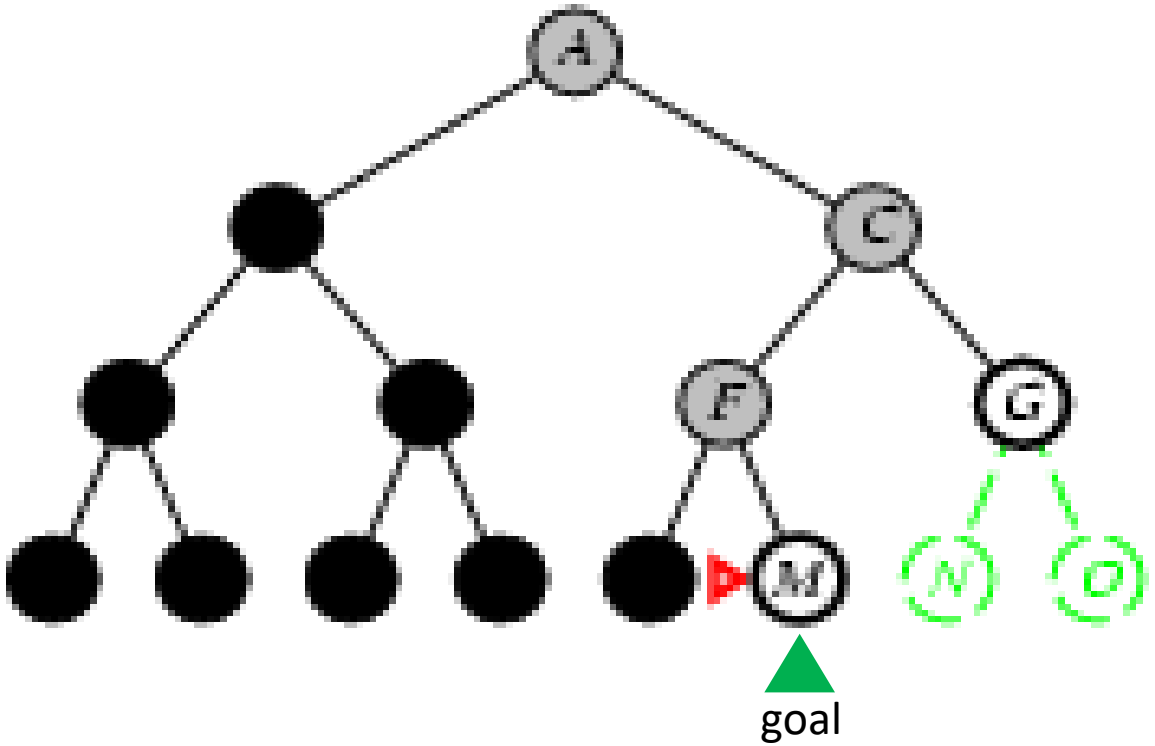
Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



Depth-first search (DFS)

- Expand the deepest unexpanded node
- Implementation:
  - *fringe* = LIFO stack, i.e., put successors at front



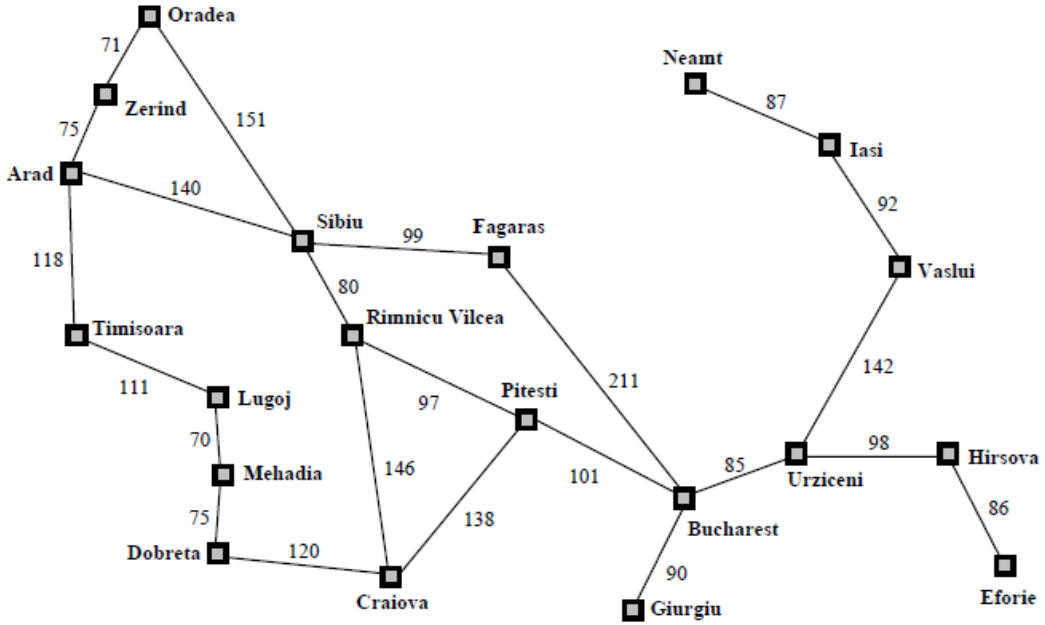
Depth-first search properties
(Tree version with goal test at node generation)

- Completeness: Fails in spaces with loops
- Time: $O(b^m)$ - m could be much larger than d
- Space: O(bm) - linear in space
- Optimal: No

b branch factor
d depth of optimal solution
m maximum length of a path in the state space (could be ∞)

Tree vs. graph search variant

- Without vs. with remembering which states have been visited already
- DFS graph search is complete if the graph is finite



Generic graph search algorithm

- Add root node (start state) to container
- Repeat
  - If container empty → FAILED
  - Choose a node to take out from the container
  - Store the state to avoid it in the future
  - If node corresponds to goal → FOUND IT!
  - Add successors / children to the container if not already visited
- Search method depends on how we add/take out node from the container

Avoid visited states

- Can speed up search.
- Must check for repeated states many times
  - Should be fast!

- Q: So how do we implement this efficiently?
  - We use a hash table, which gives us very fast lookups. Typically, constant time!

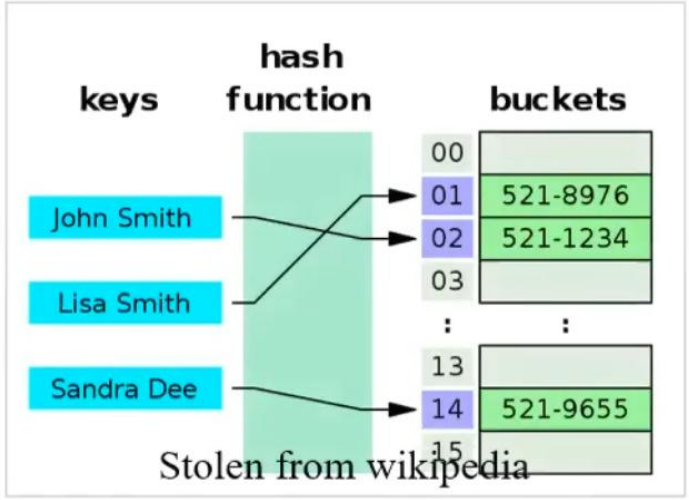
Hash table

- Want to find a way to know which of all previously visited states we should compare the new state to, so we do not have to compare to all
- Use hash function to map the state (“key”) into an index
- Often implemented as

$$index = hash_function(key) \% N$$

where N is the number of buckets or bins and % is modulo, i.e. maps to index [0, N-1]

- Lots of hash functions have been suggested
  - For example, Zobrist hashing common in games



BFS vs. DFS properties

BFS

- Completeness: Yes
- Time: $O(b^d)$
- Space: $O(b^d)$
- Optimality: Not in general

DFS

- No *(yes for graph search on a finite graph)*
- $O(b^m)$
- O(bm)
- No

b branch factor
d depth of optimal solution
m maximum length of a path (can be ∞)

DFS variation: Depth-limited search

- DFS with depth limit l
  - Do not expand nodes at depth l
- If we pick l < d we are in trouble
- How to set the depth limit if the depth is unknown?

DFS variation: Iterative deepening search (IDS)

- Idea: Iterate depth-limited search with increasing depth limit.

Iterative deepening search *l* =0

Limit = 0

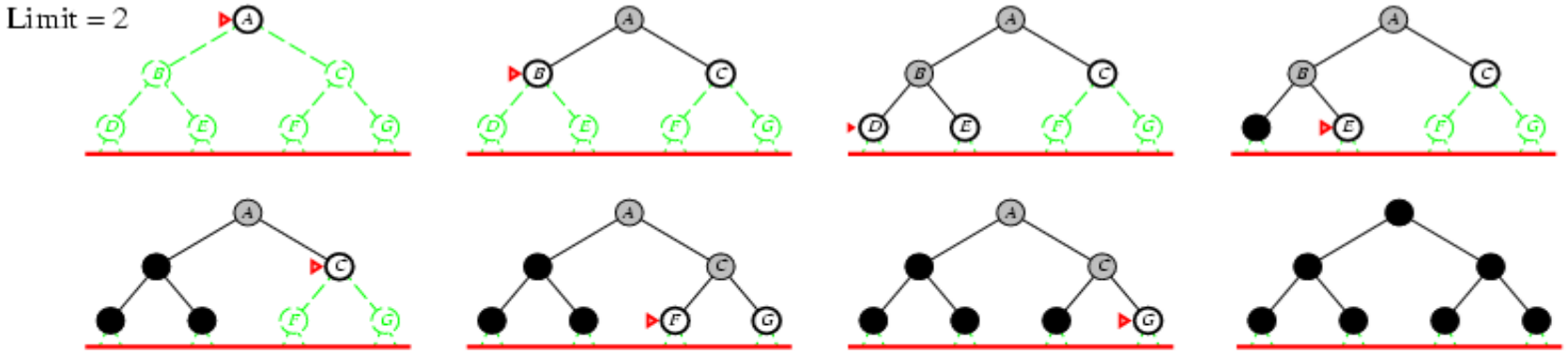




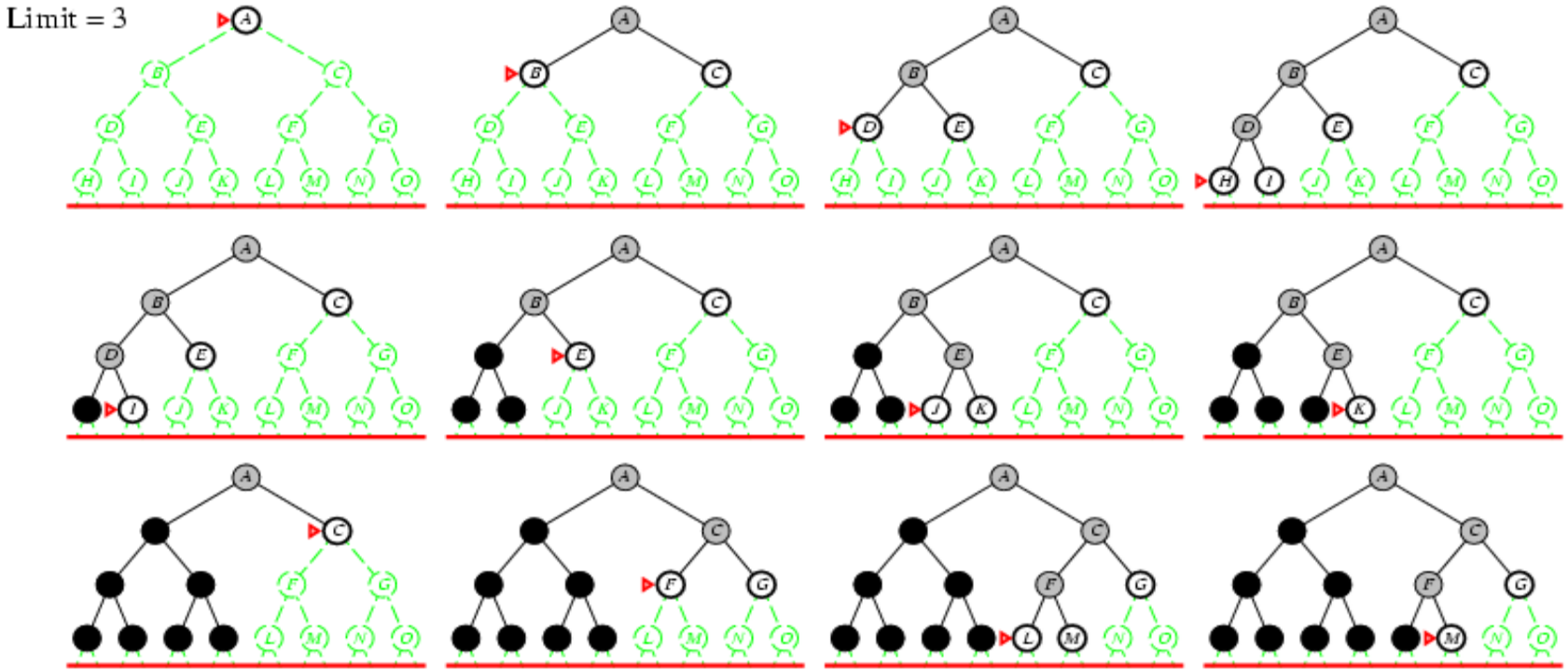
Iterative deepening search *l* =1



Iterative deepening search *l* =2



Iterative deepening search *l* =3



Iterative deepening search (IDS)

- Does this not lead to a massive waste of time?

Iterative deepening search

- Number of nodes generated in a depth-limited search to depth *d* with branching factor *b*:

$$N_{DLS} = b^0 + b^1 + b^2 + \ldots + b^{d-2} + b^{d-1} + b^d$$

- Number of nodes generated in an iterative deepening search to depth *d* with branching factor *b*:

$$N_{IDS} = (d+1)b^0 + d\,b^1 + (d-1)b^2 + \ldots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

- For *b = 10*, *d = 5*,
  - $N_{DLS} = 1 + 10 + 100 + 1{,}000 + 10{,}000 + 100{,}000 = 111{,}111$
  - $N_{IDS} = 6 + 50 + 400 + 3{,}000 + 20{,}000 + 100{,}000 = 123{,}456$

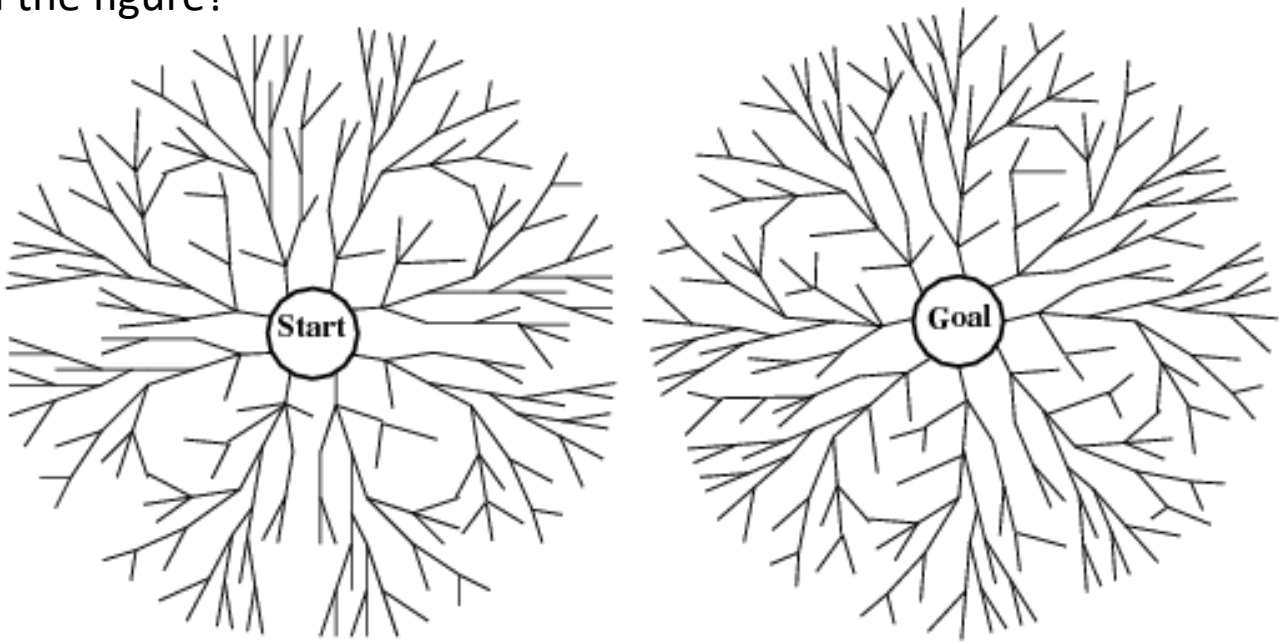
- Overhead = (123,456 - 111,111)/111,111 = 11%

Properties of iterative deepening search
(Tree version with goal test at node generation)

- Completeness: Yes
- Time: $(d + 1)b^0 + db^1 + (d - 1)b^2 + ... + b^d = O(b^d)$
- Space: O(bd)
- Optimal: under the same conditions as BFS

Bi-directional search

- Motivation: $b^{d/2}+b^{d/2} << b^{d}$
- How do we see this in the figure?



The area of the two circles much smaller than one circle with twice the radius

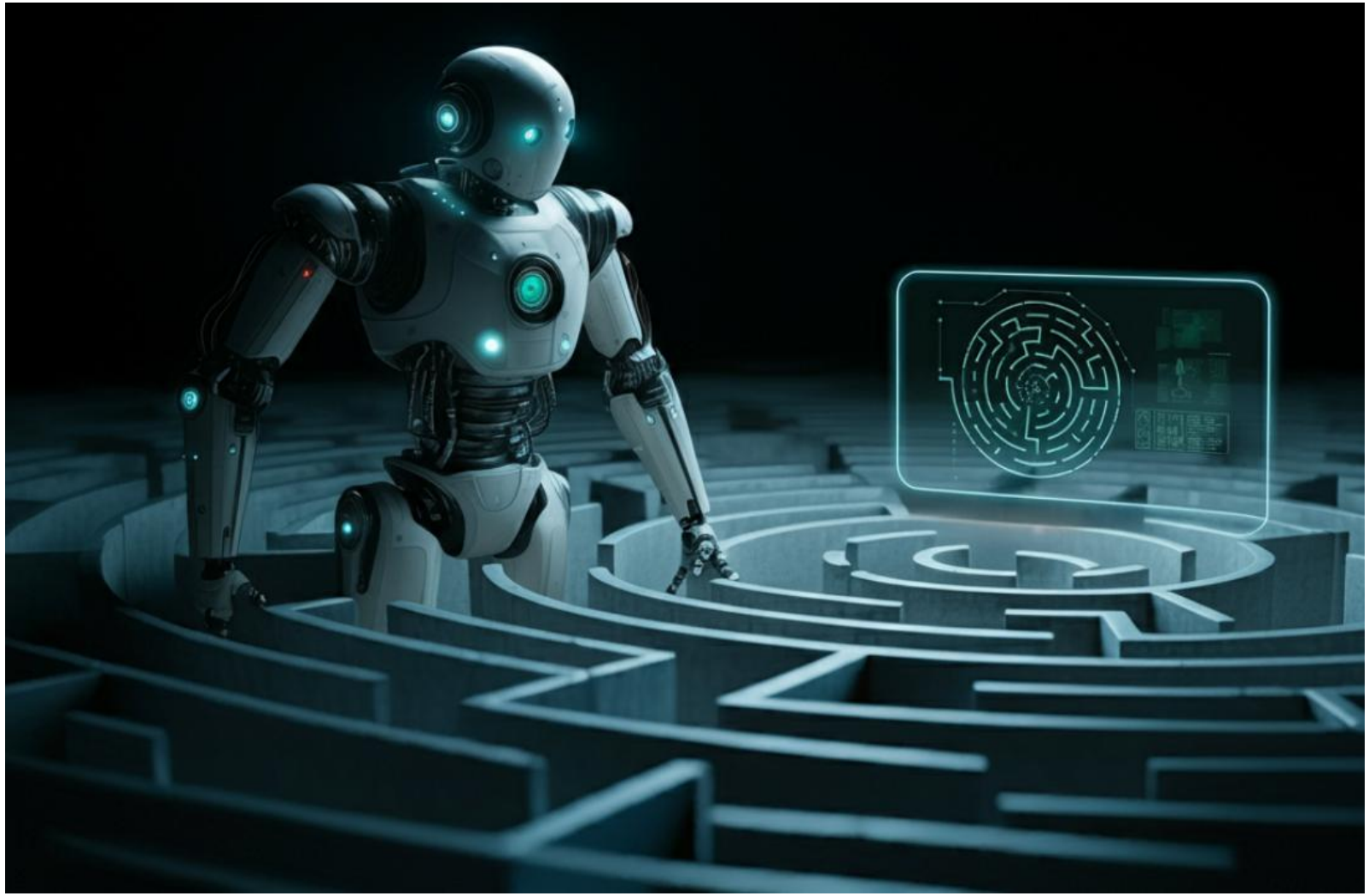
Bi-directional search

- Search in parallel
  - start → goal
  - goal → start
- Look for them to meet
- Why may this be difficult?
  - Need predecessor function and not just successor
  - Need goal states not just goal test

Break



Informed search



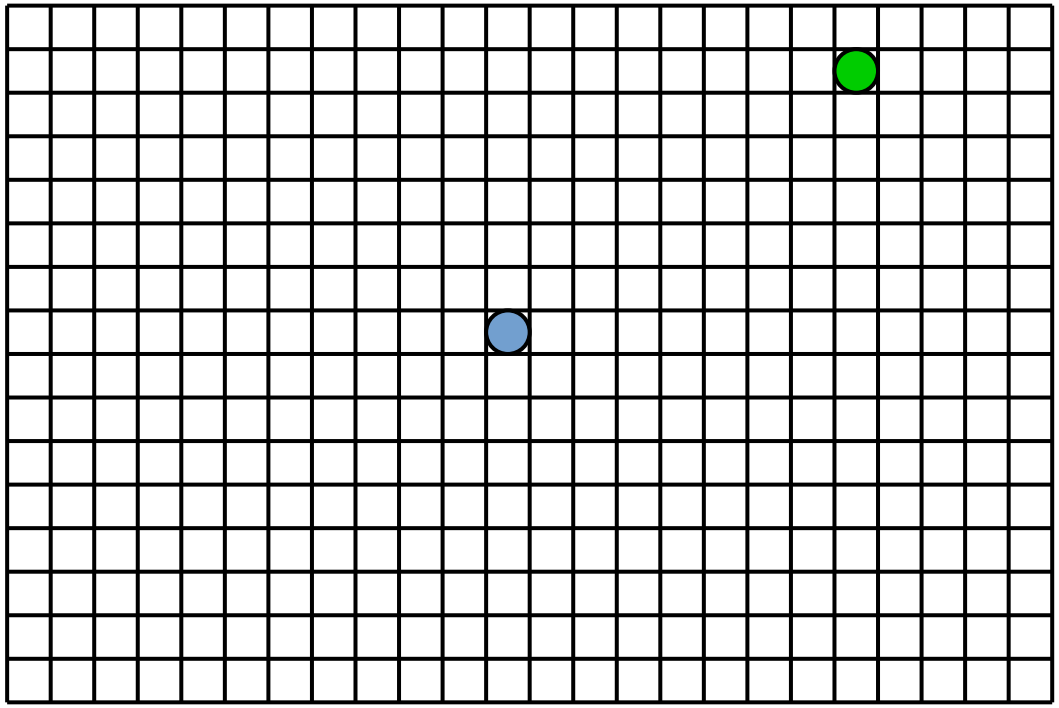
Informed search and exploration

- Un-informed search (e.g., BFS and DFS) often very inefficient
- Will look at two ways to improve
  - Integrate task knowledge into the search
  - Use information for local search

- We want to make the expand function cleverer, i.e., which node to look at next
- Ideally: Expand nodes heading directly for the goal and no others

Grid example

- Imagine a large grid
- Want to find a path to a cell 14 cells away

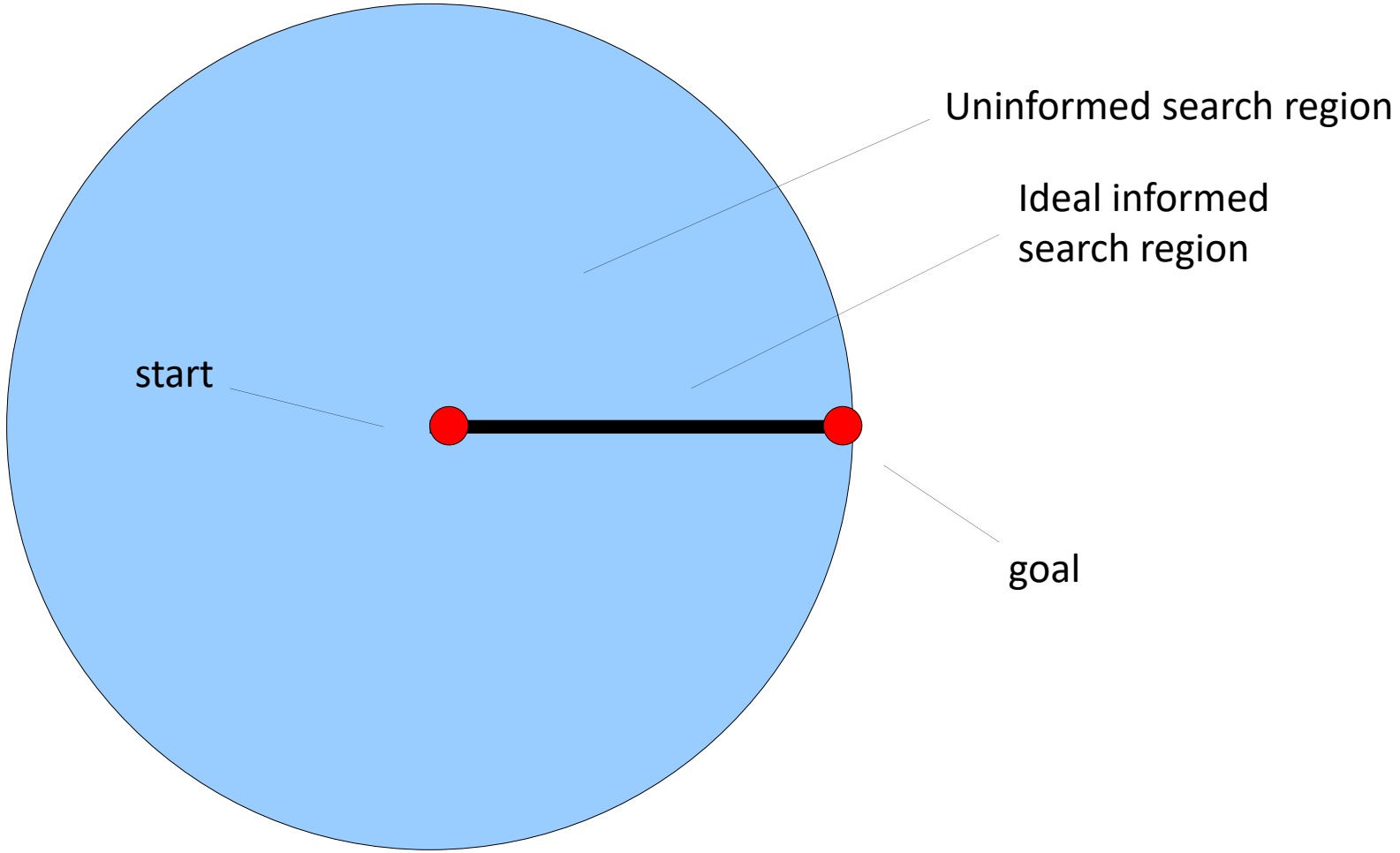


Grid example cont'd

- We want something that makes us expand nodes in the direction of the goal!
- Ideally, we could get down to 14 nodes expanded
  - BFS naïve: ~1.000.000.000 ($O(b^{d+1})$, b=4)
  - BFS avoid immediate backtracking: ~10.000.000
  - BFS avoid visited states: ~500 (area of square)
  - "Optimal" informed: 14

- A lot to gain!

Uninformed vs informed search



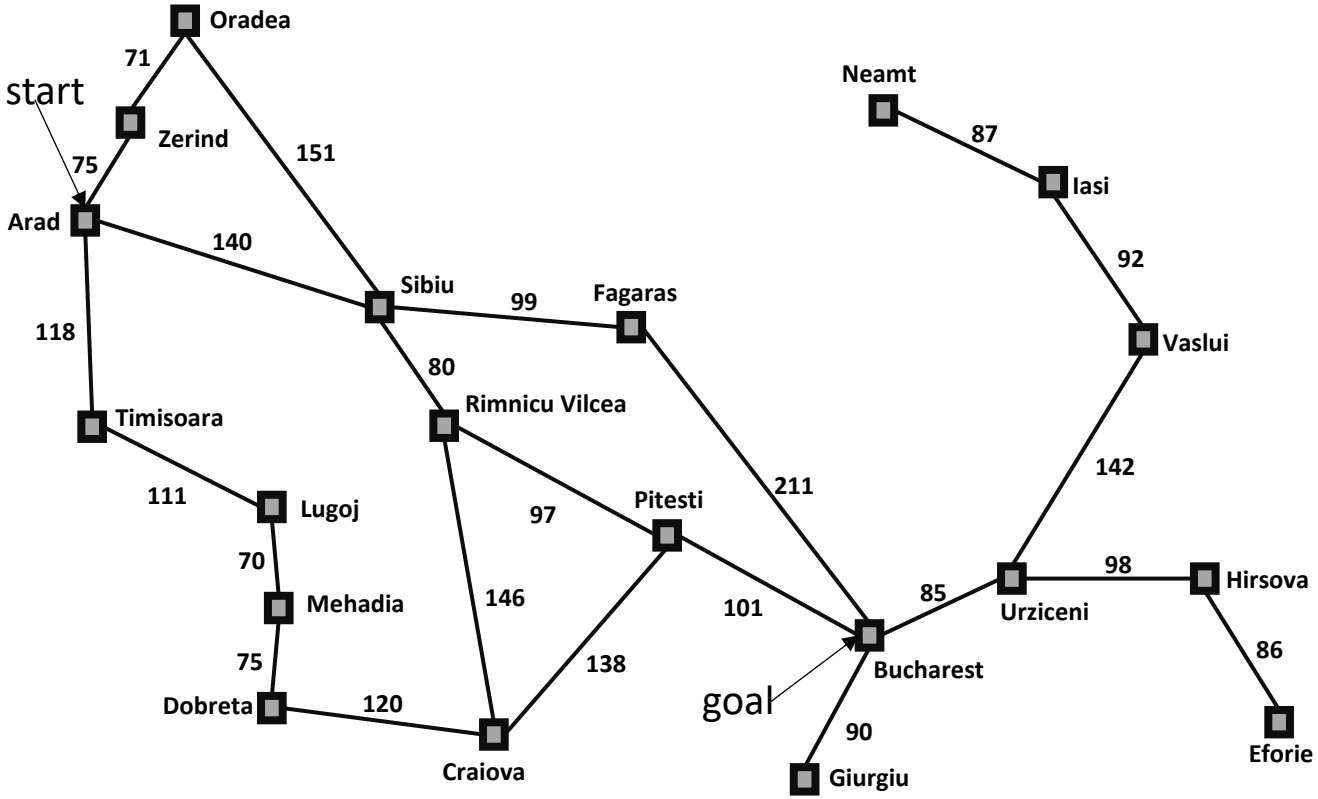
Informed search

- Often, we have more information than
  - Initial state
  - Successor function
  - Goal test
  - Path cost

- We know something about the problem
- Use this to direct the search towards the goal

Example

- Where would you go first on the way from Arad to Bucharest? Why?



Informed search strategies

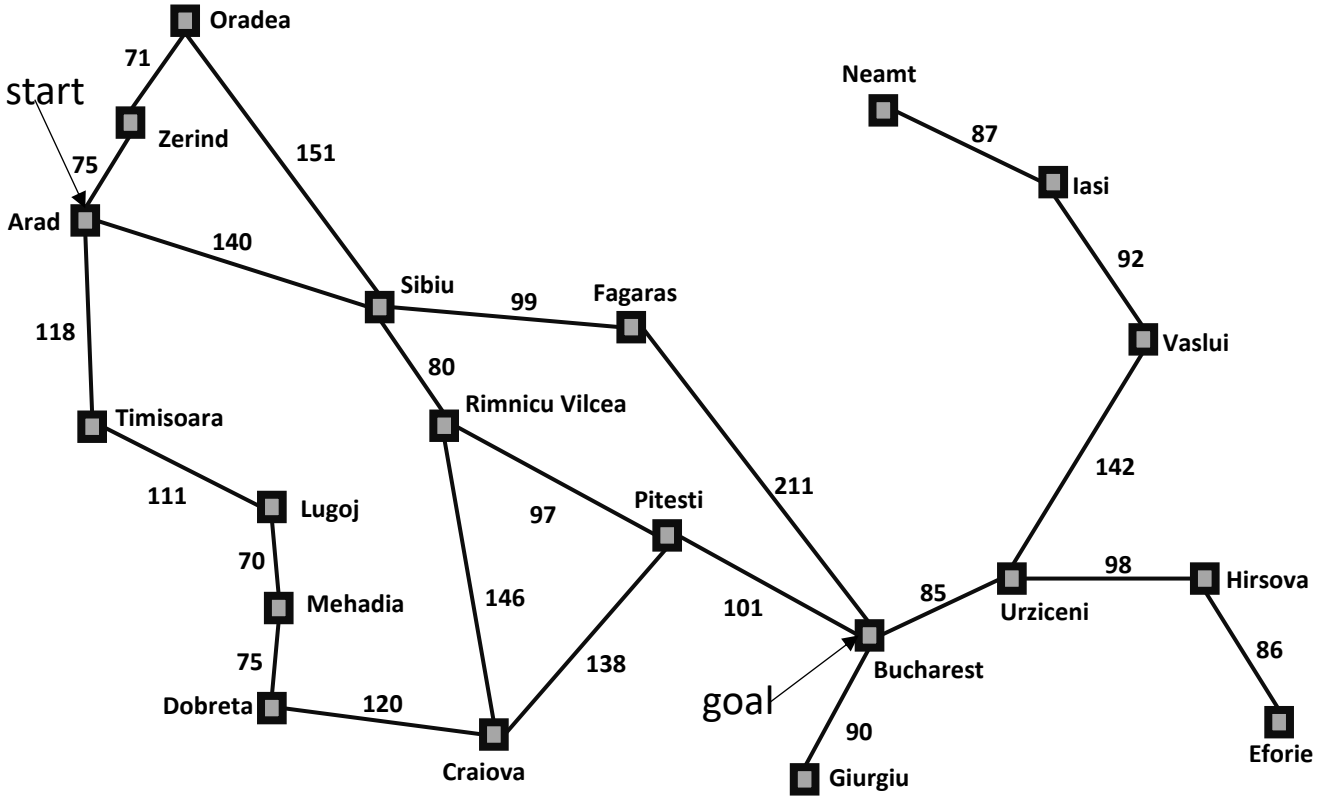
- General form: “Best First Search"
- Expand low-cost nodes before higher cost nodes
- Use evaluation function f(n) to estimate cost

A first go at the evaluation function

- Design of a heuristic function h(n)
  - Estimated cost of the cheapest path from node n to goal

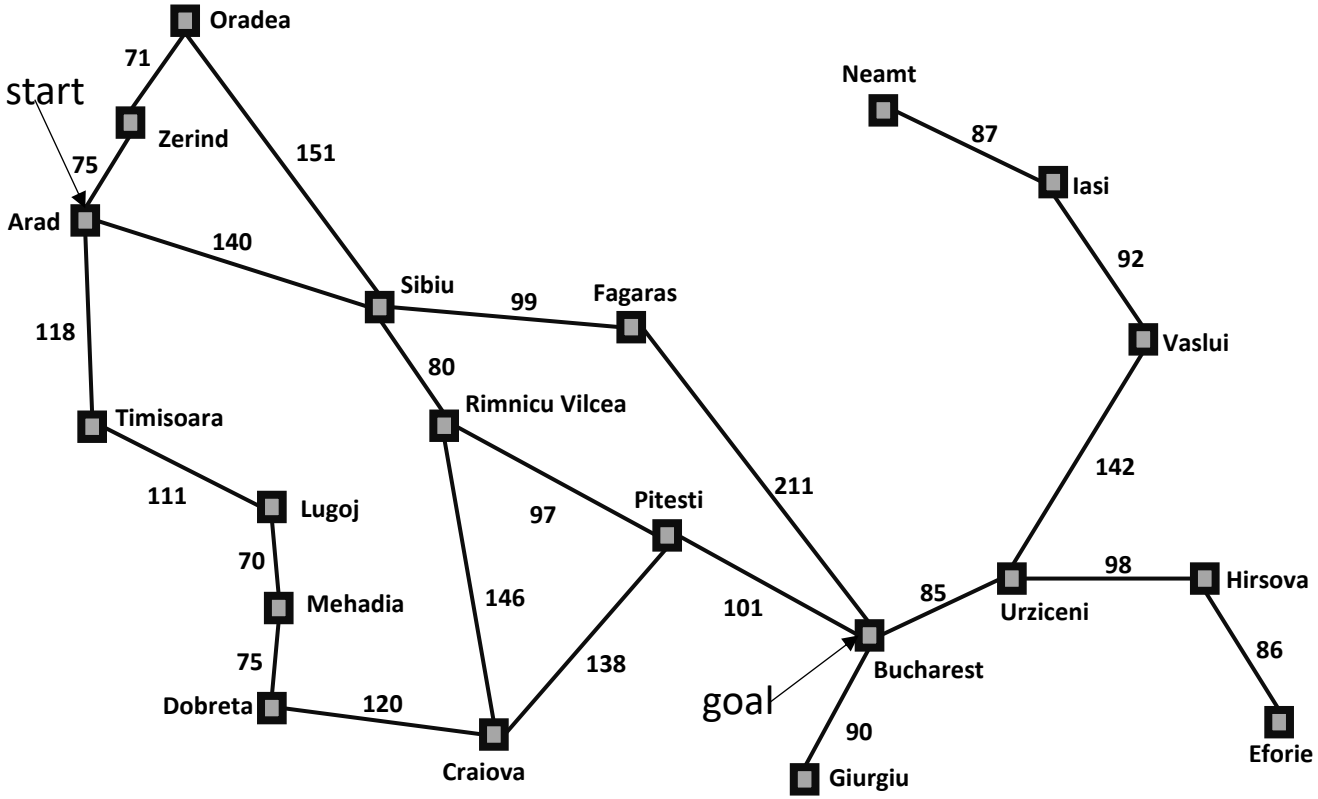
Example

- What would you pick for h(n) in this case?



Example

- Straight line distance from current city to Bucharest as h(n)



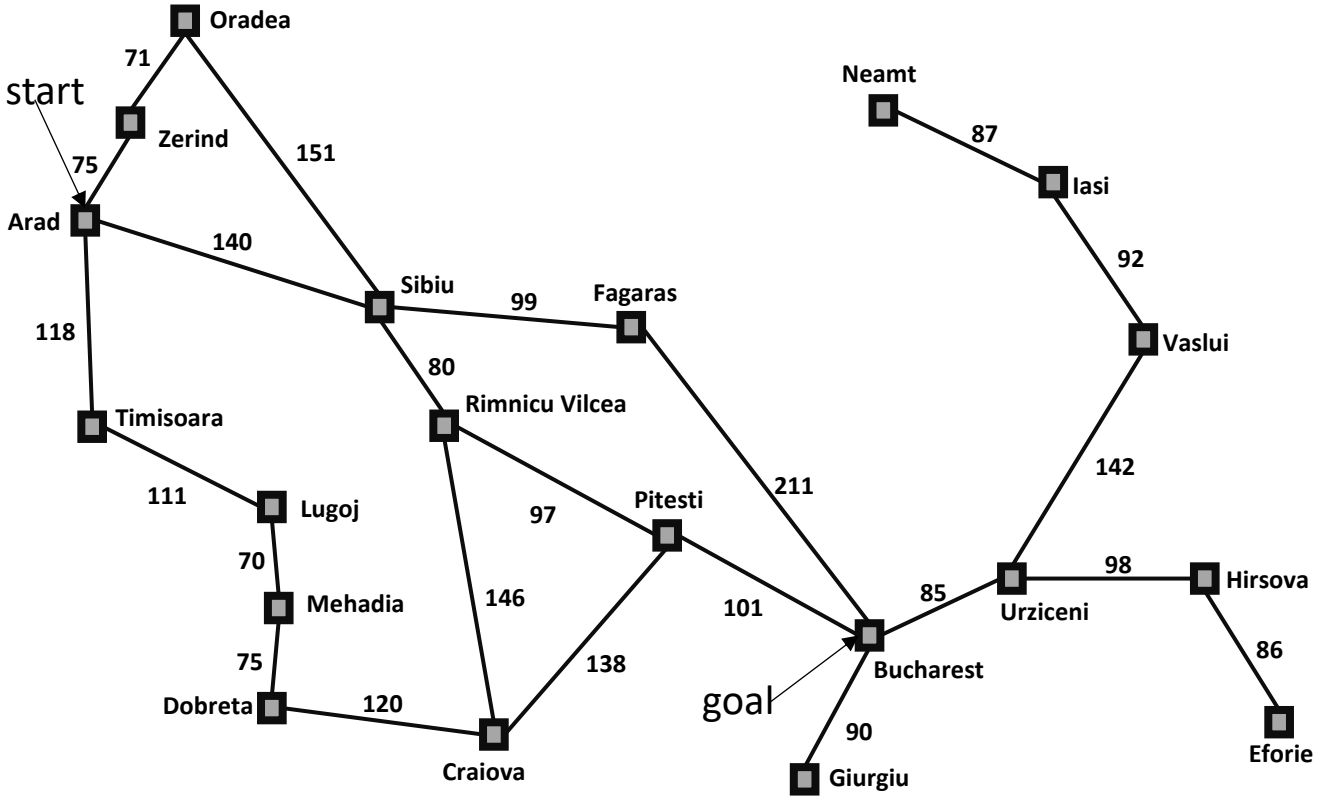
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search

- Let h(n) be a heuristic for the search
  - Ex: $h_{SLD}(n)$ - straight line distance from n to Bucharest
- Greedy search uses f(n) = h(n) for expansion of nodes that appear to be closest to the goal.

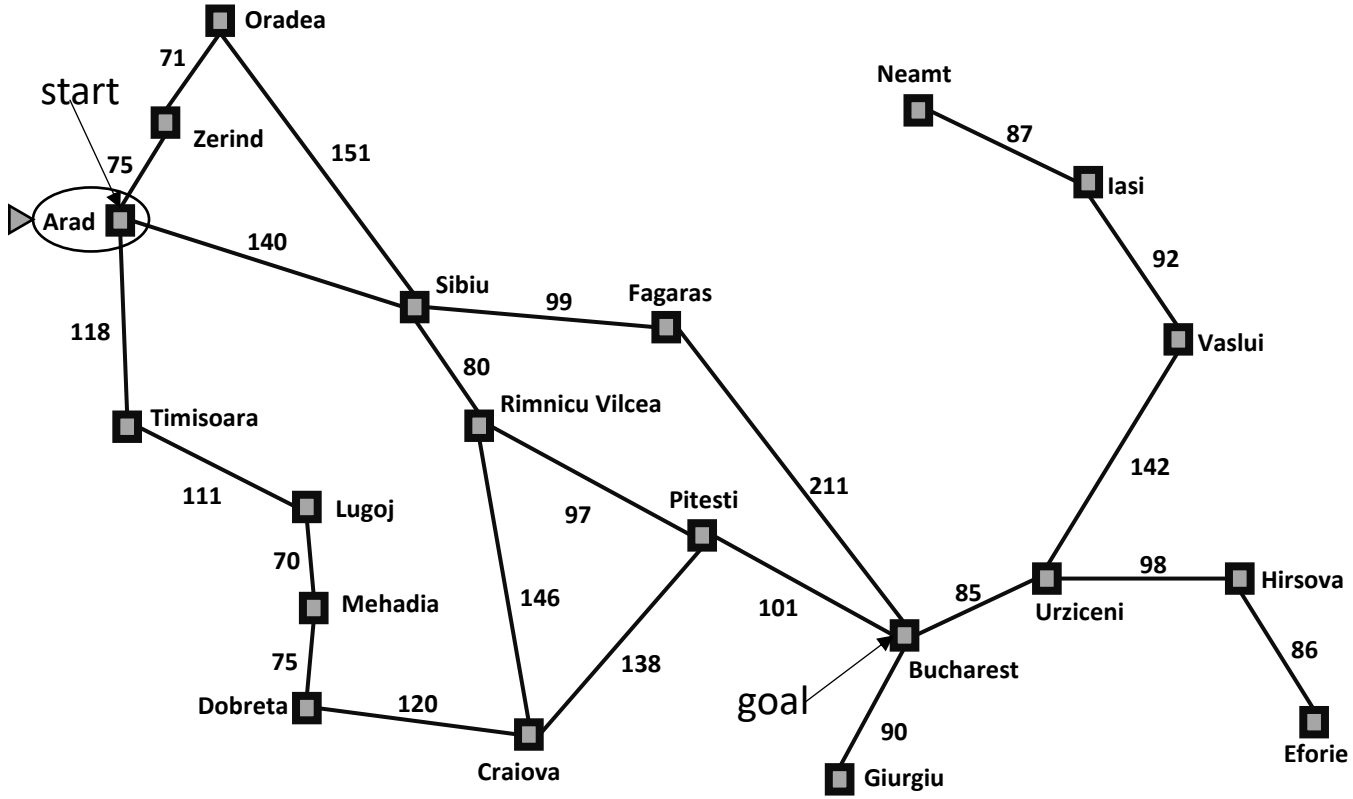
Question

- What path would greedy best-first find to Bucharest? Is it optimal?



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Expansion greedy best-first search

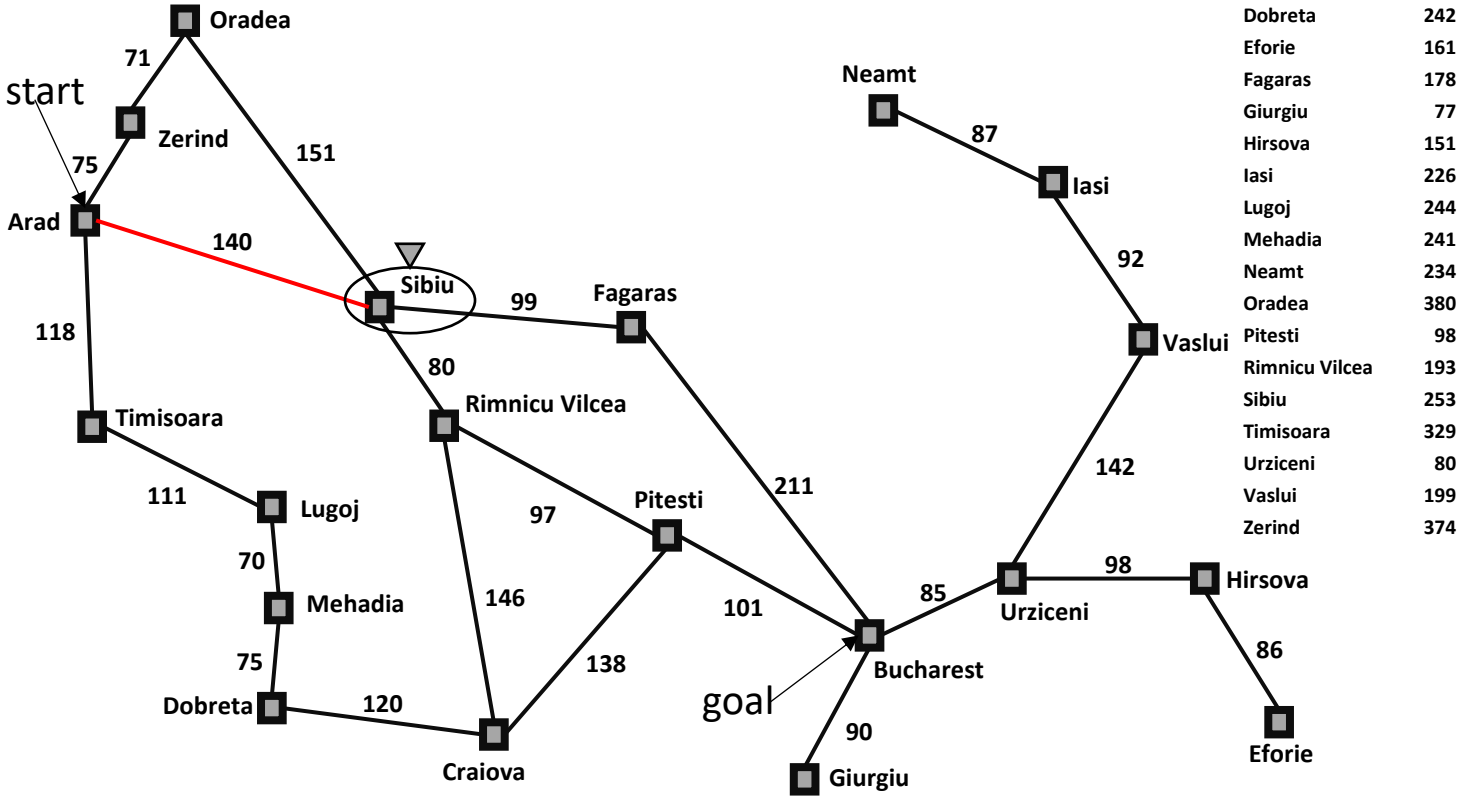




Straight-line distance to Bucharest

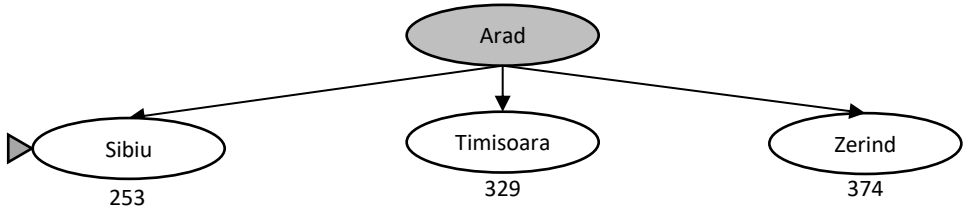
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Expansion greedy best-first search

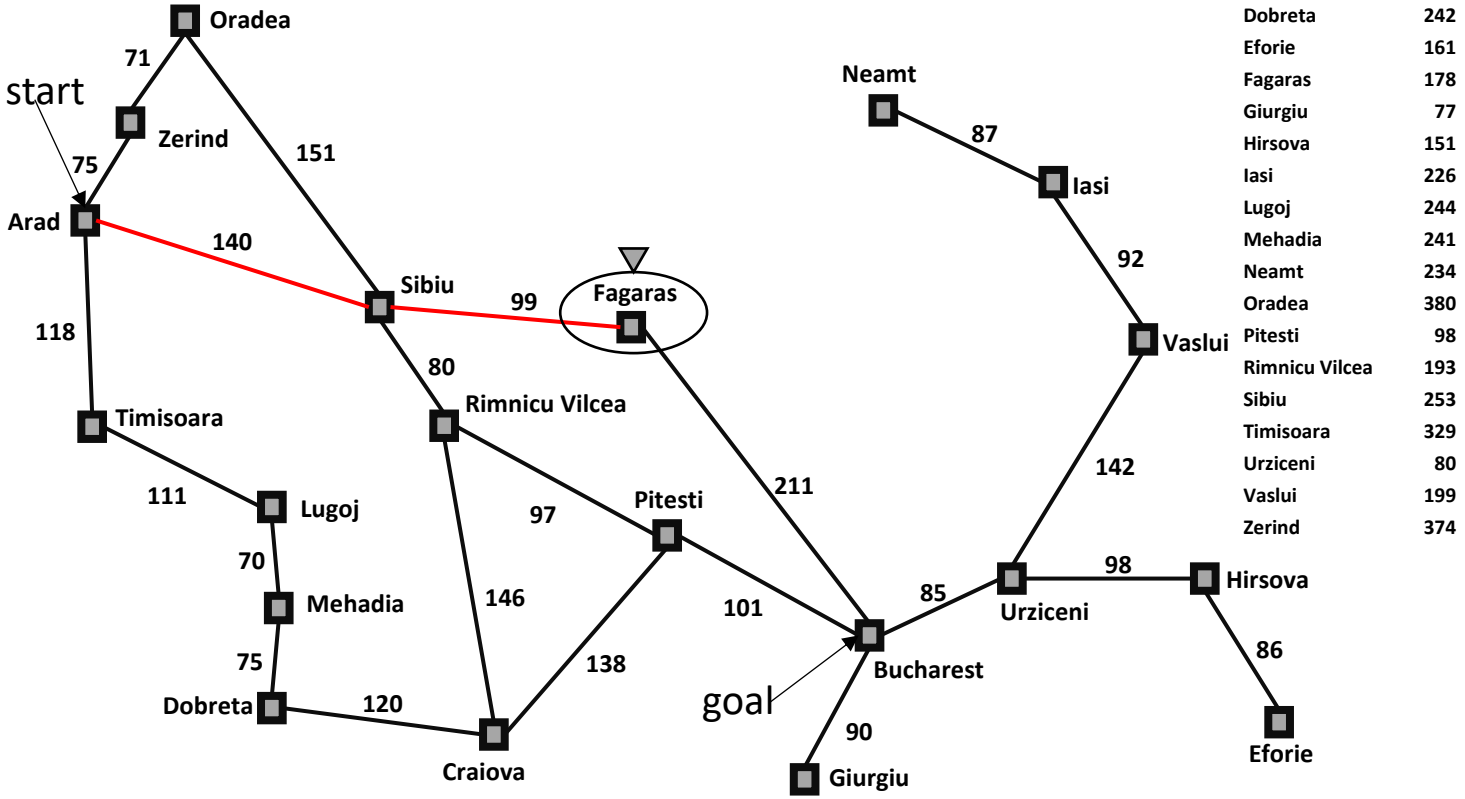


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

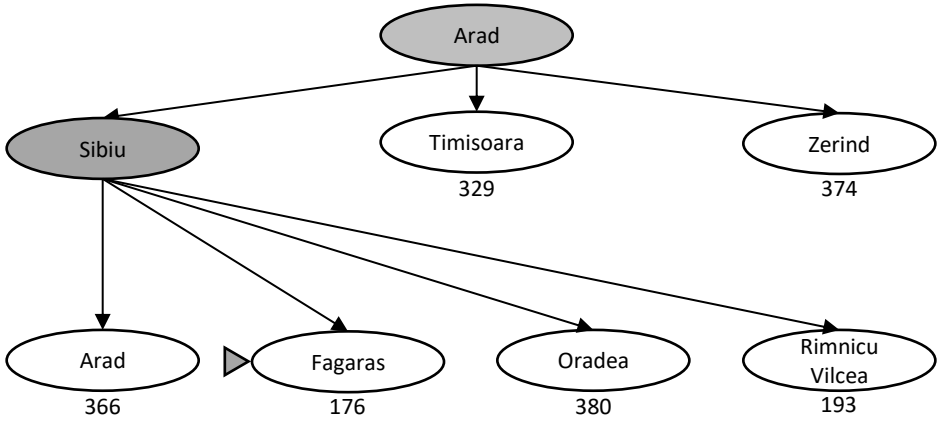


Expansion greedy best-first search

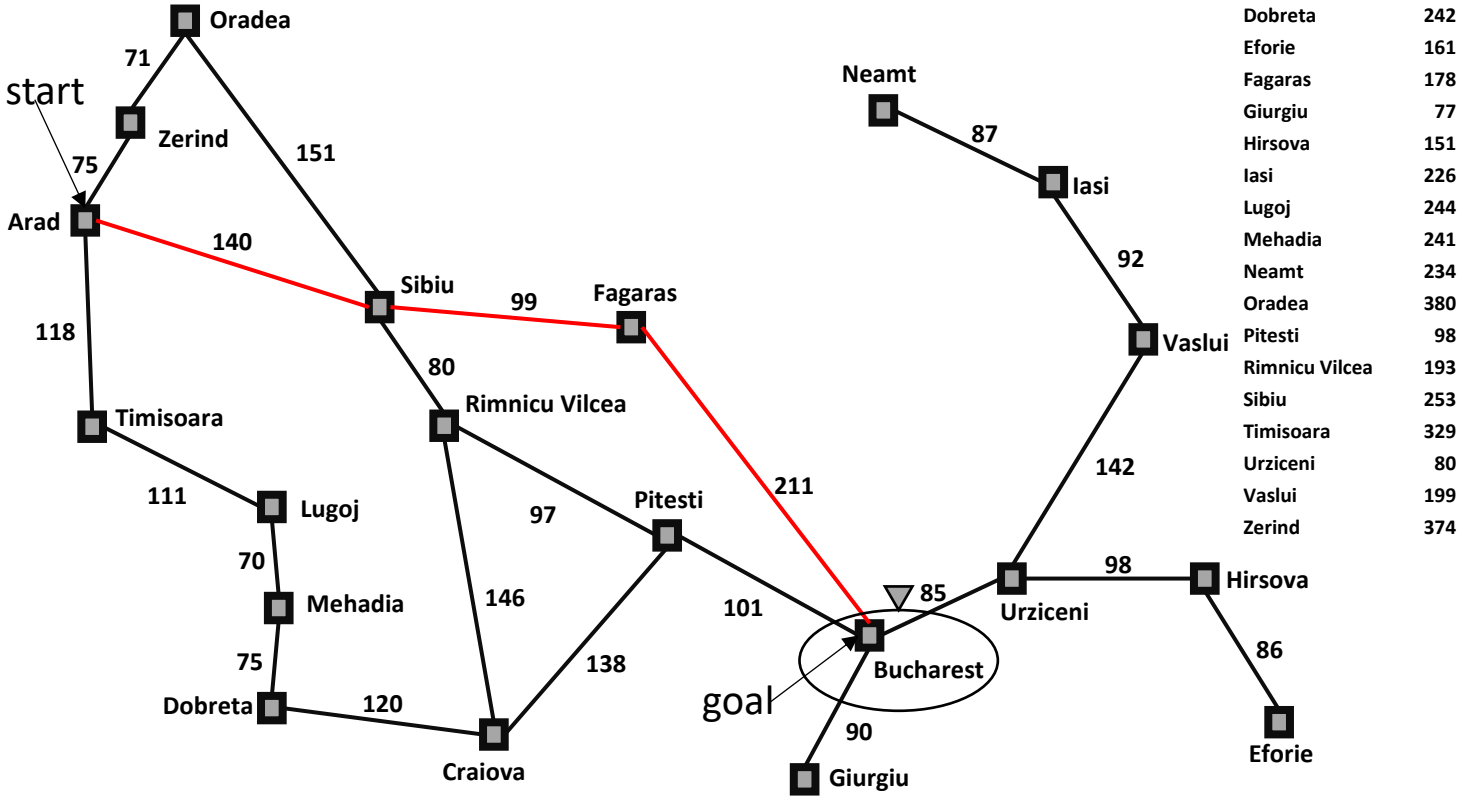


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

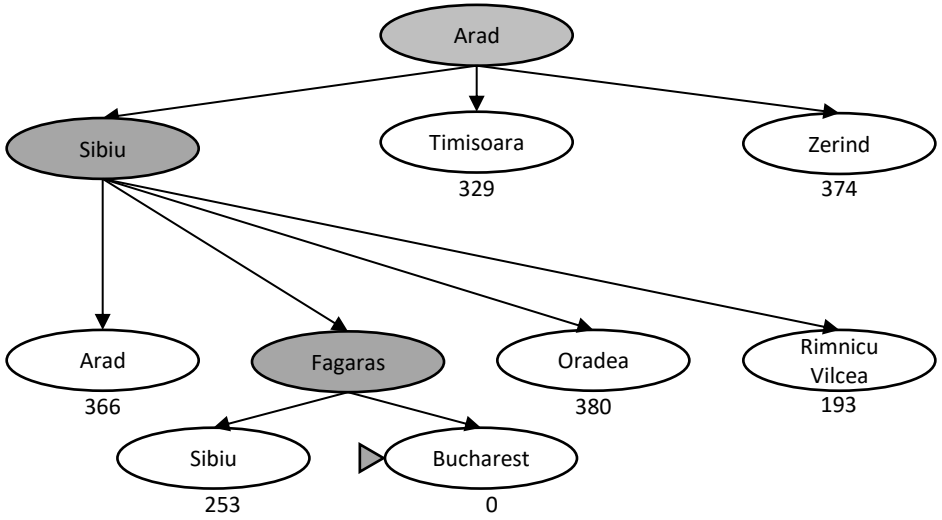


Expansion greedy best-first search



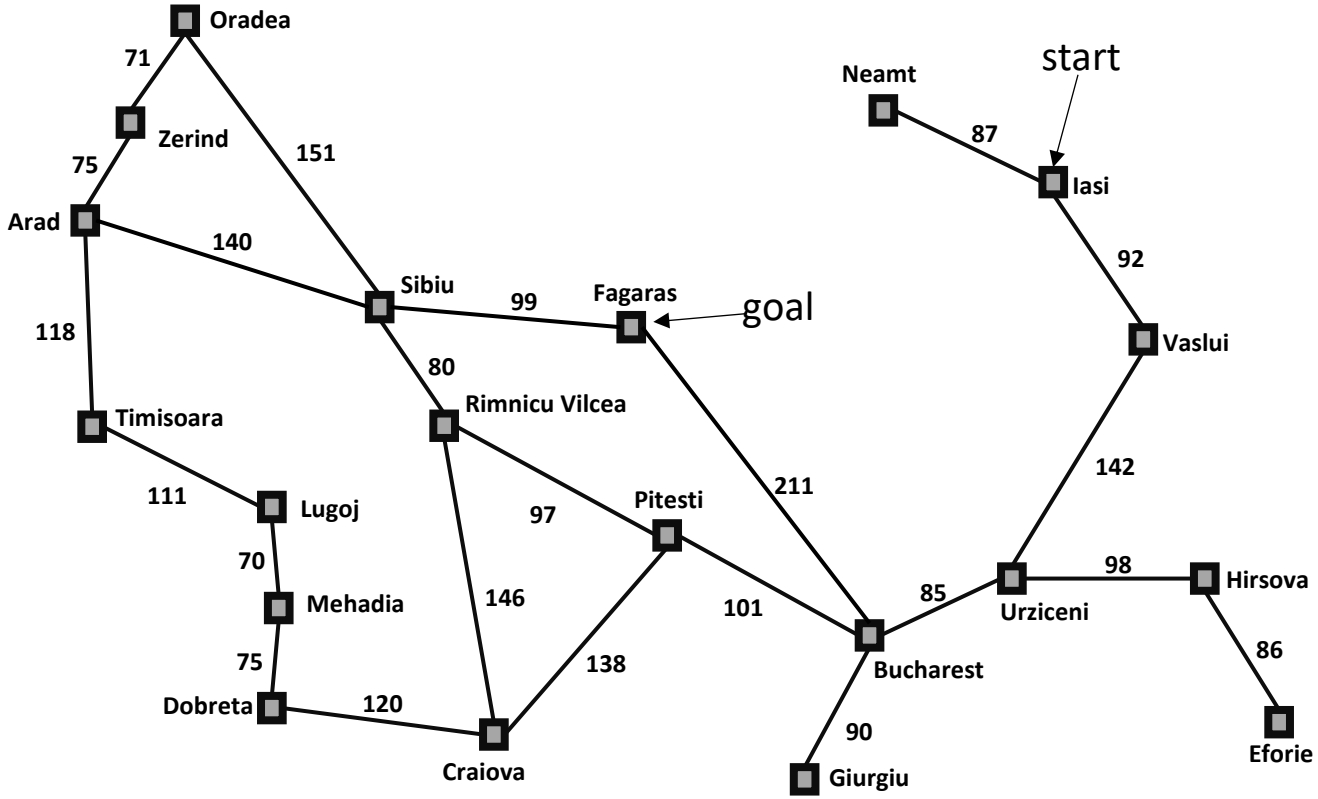
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Question

- Which path would a tree greedy best-first search (i.e. one without repeated state checking) take from Iasi to Fagaras?



Properties of greedy best-first search

- Completeness: Could get stuck in loops; repeated-state checking needed to ensure completeness
- Time: $O(b^m)$ - a good heuristic can improve this significantly in practice
- Space: $O(b^m)$ - keep all nodes in memory
- Optimal: no

How can we make it better?

- Greedy best-first search ignores the past and only looks at what seems to be the best from where we are now
- Would be good to take cost so far into account

A* search

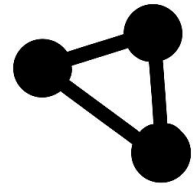
- Minimizing the total estimated solution cost
- Still widely used best-first method
  - Robot Navigation and Planning
  - Autonomous Driving
  - Game AI
  - Warehouse Logistics
  - Drone Flight
- Evaluation function f (n) = g(n) + h(n)
  - g(n) the cost from start to node n, i.e. cost thus far
  - h(n) estimate of cost to get to goal
  - f (n) estimated total cost of reaching the goal from the start via node n



Robot Navigation



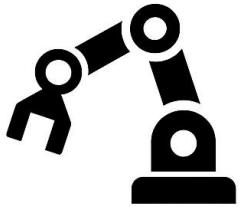
Autonomous Driving



Game AI / NPC Pathfinding



Warehouse Robots / AGVs



Robot Arm Motion Planning



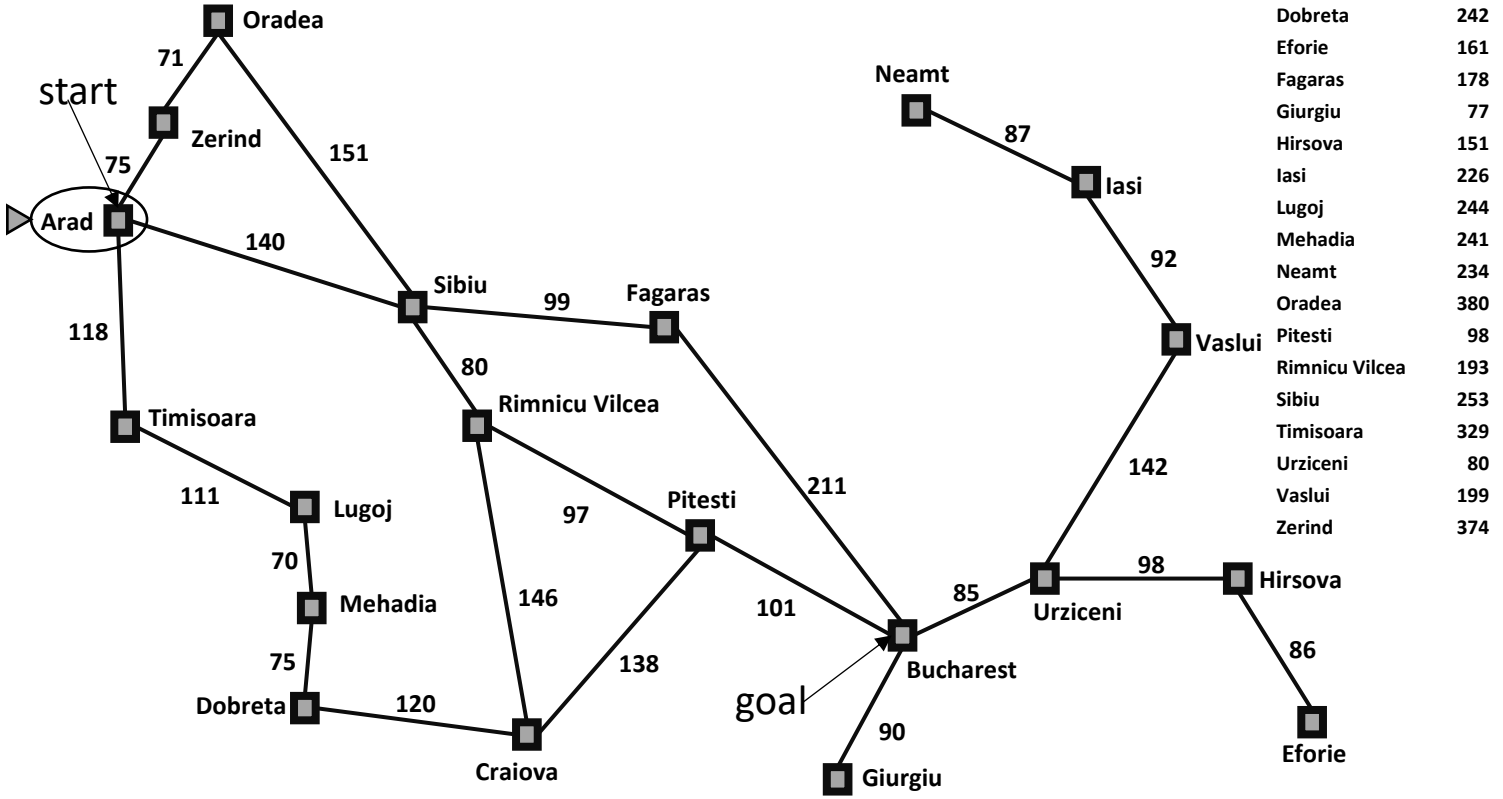
Drone Flight

A* and optimality

- A* is optimal if h(n) is an admissible heuristic
  - h(n) admissible if it does not overestimate the cost

Question

- What path would A* find from Arad to Bucharest using the straight line heuristic?

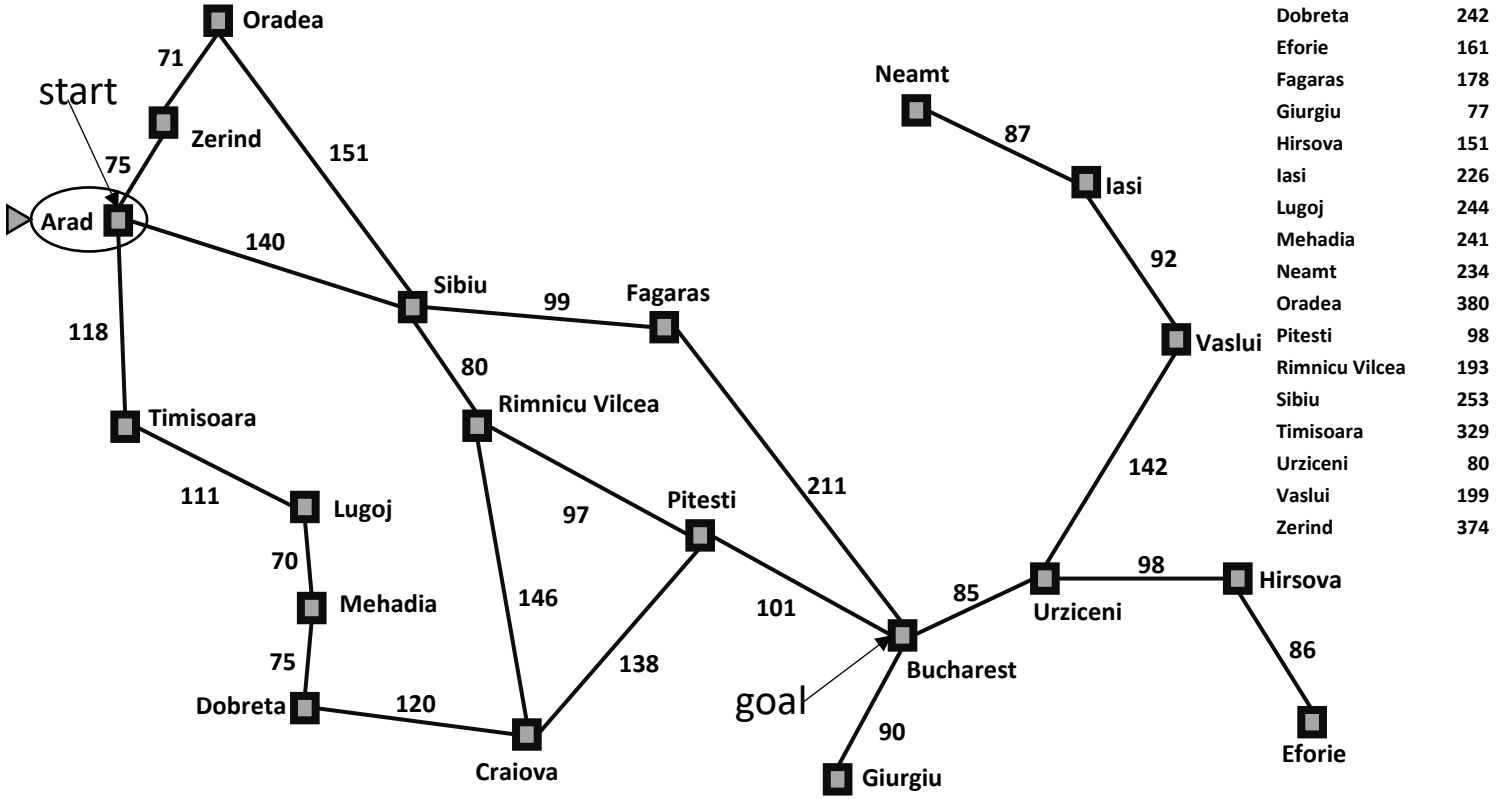


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

g(n) and h(n)

- g(n) is distance traveled so far
- h(n) is the straight line distance

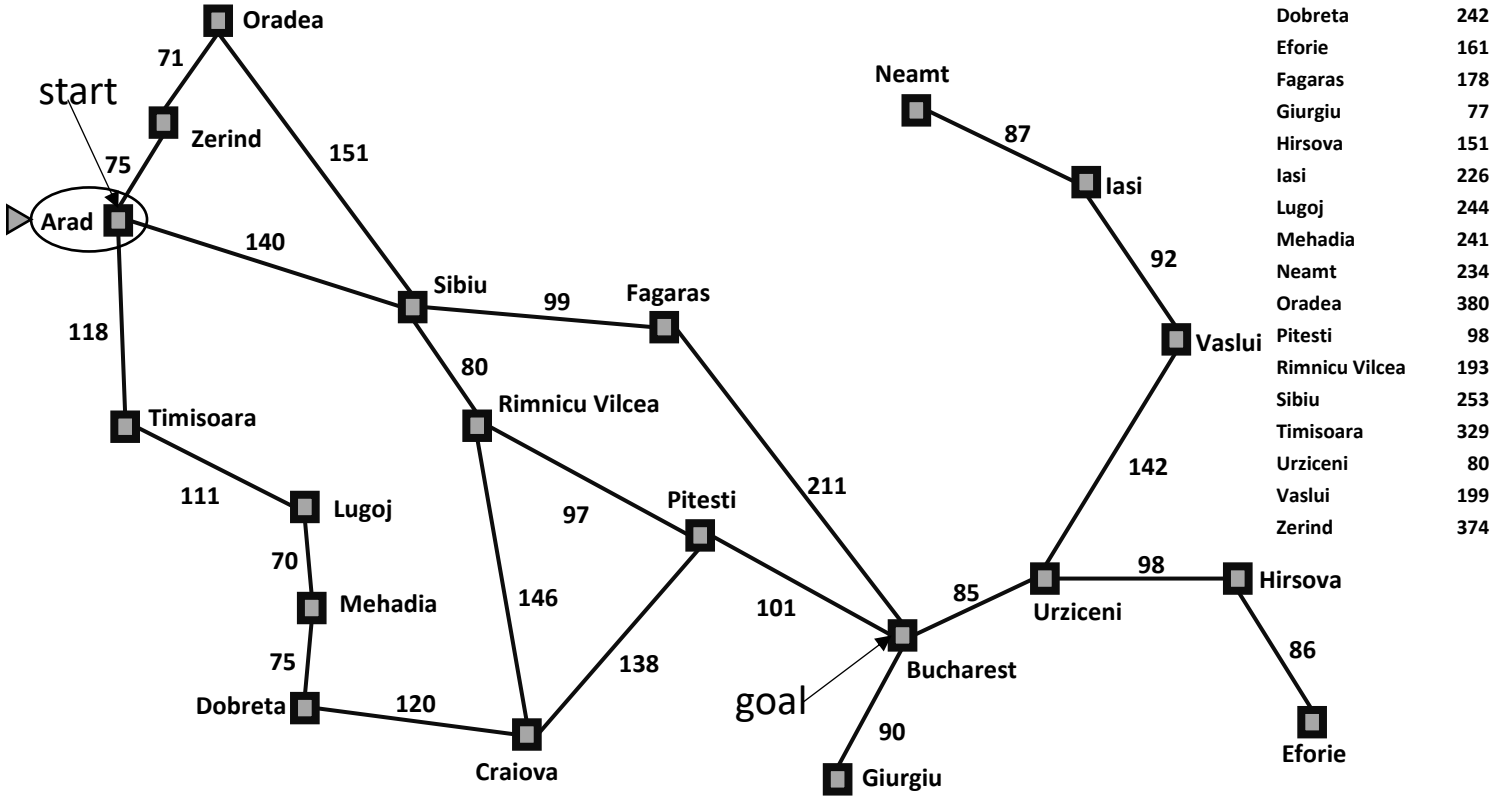


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

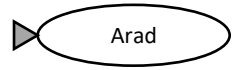
Node expansion in A*

- f(n) = g(n) + h(n)



Straight-line distance to Bucharest

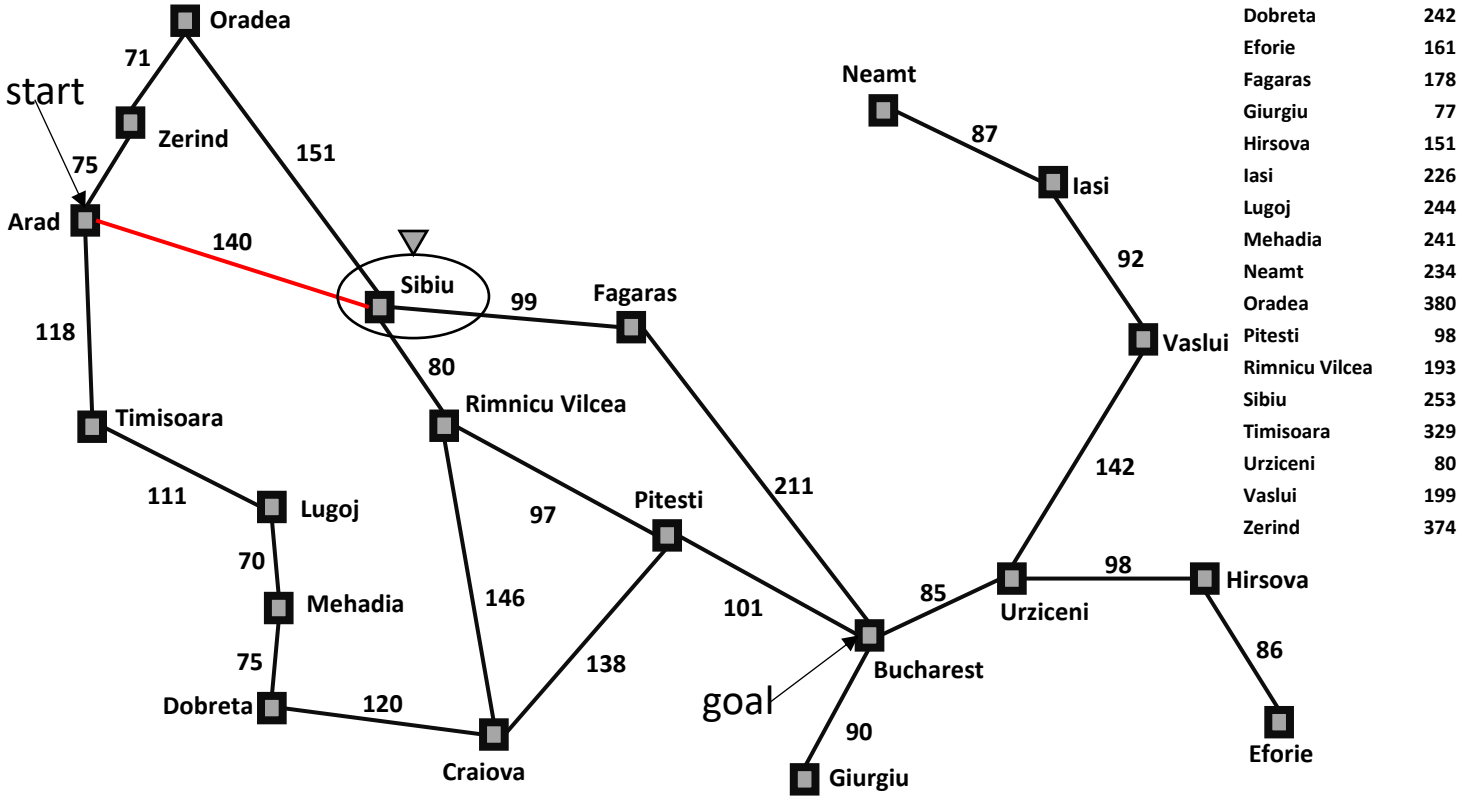
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



0 + 366 = 366

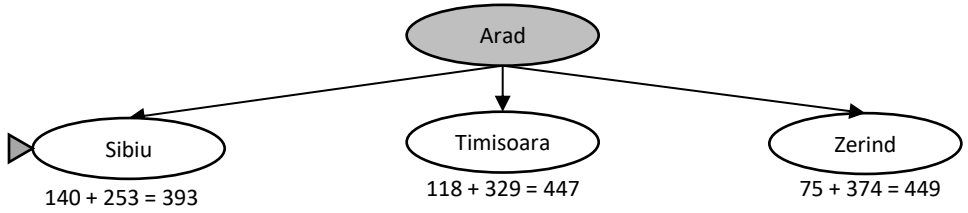
Node expansion in A*

- f(n) = g(n) + h(n)



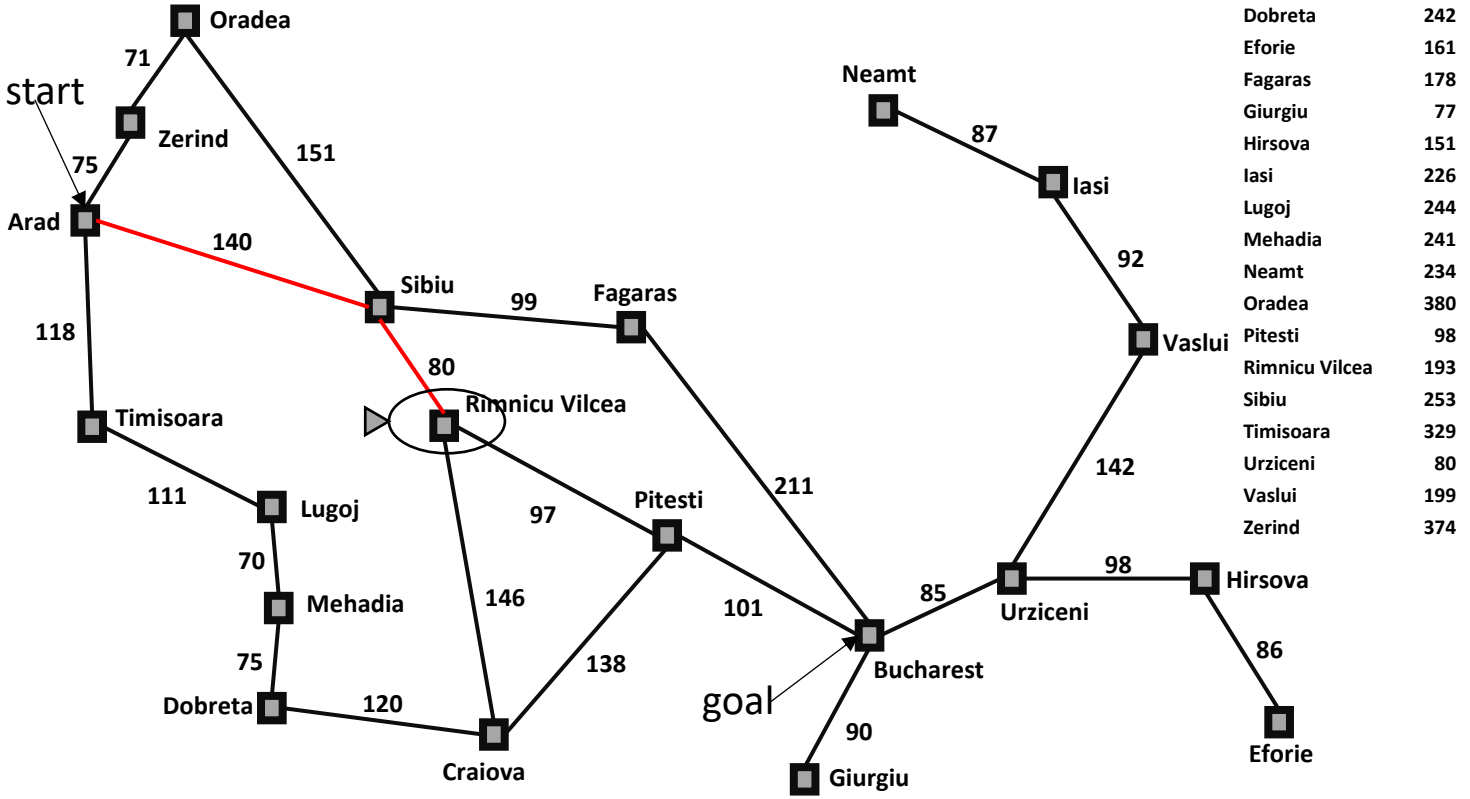
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



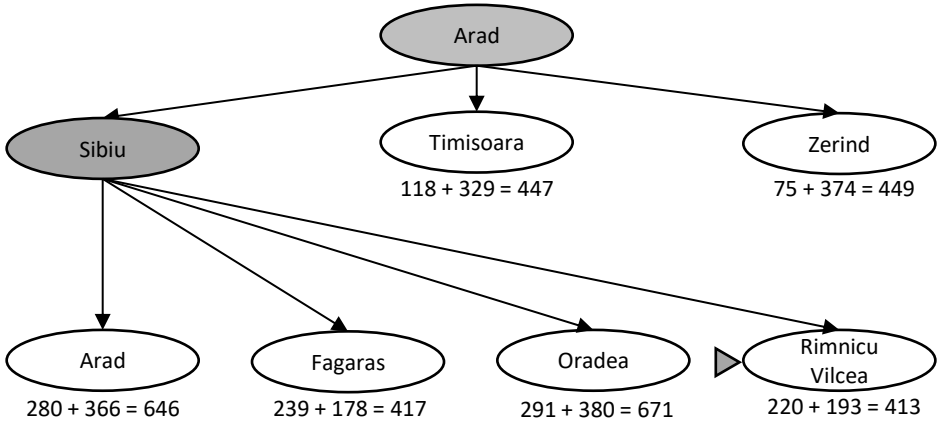
Node expansion in A*

- f(n) = g(n) + h(n)



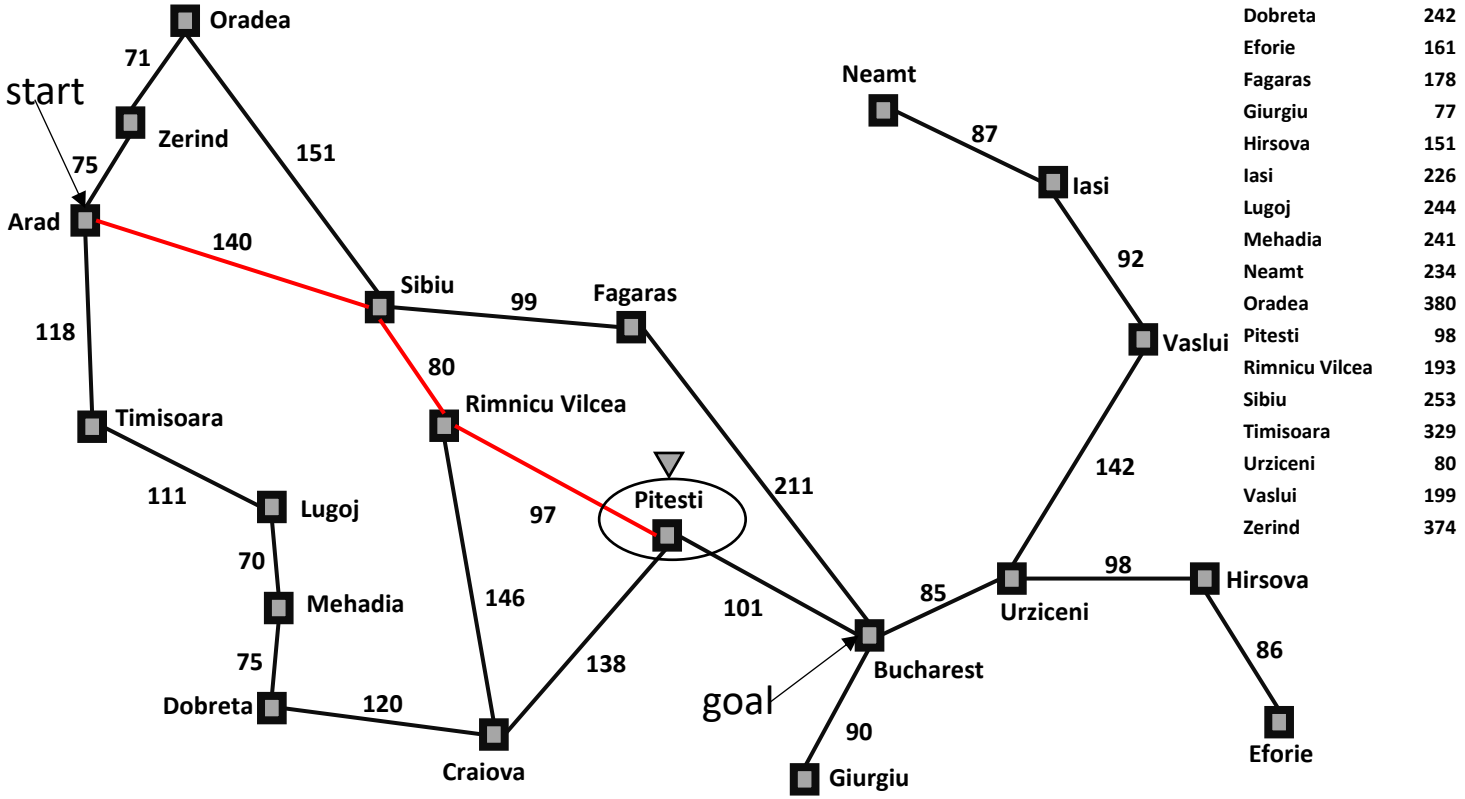
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



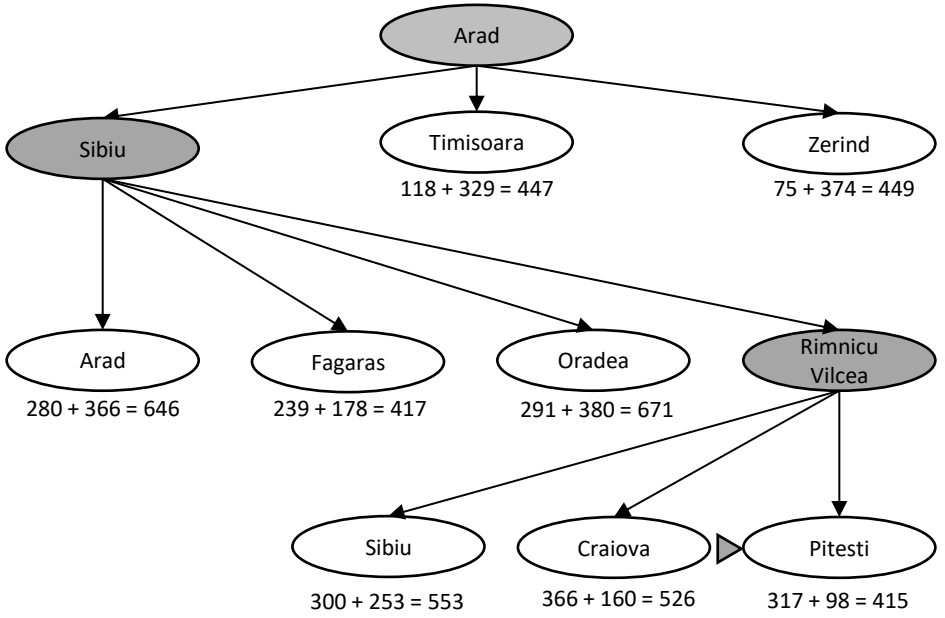
Node expansion in A*

- f(n) = g(n) + h(n)



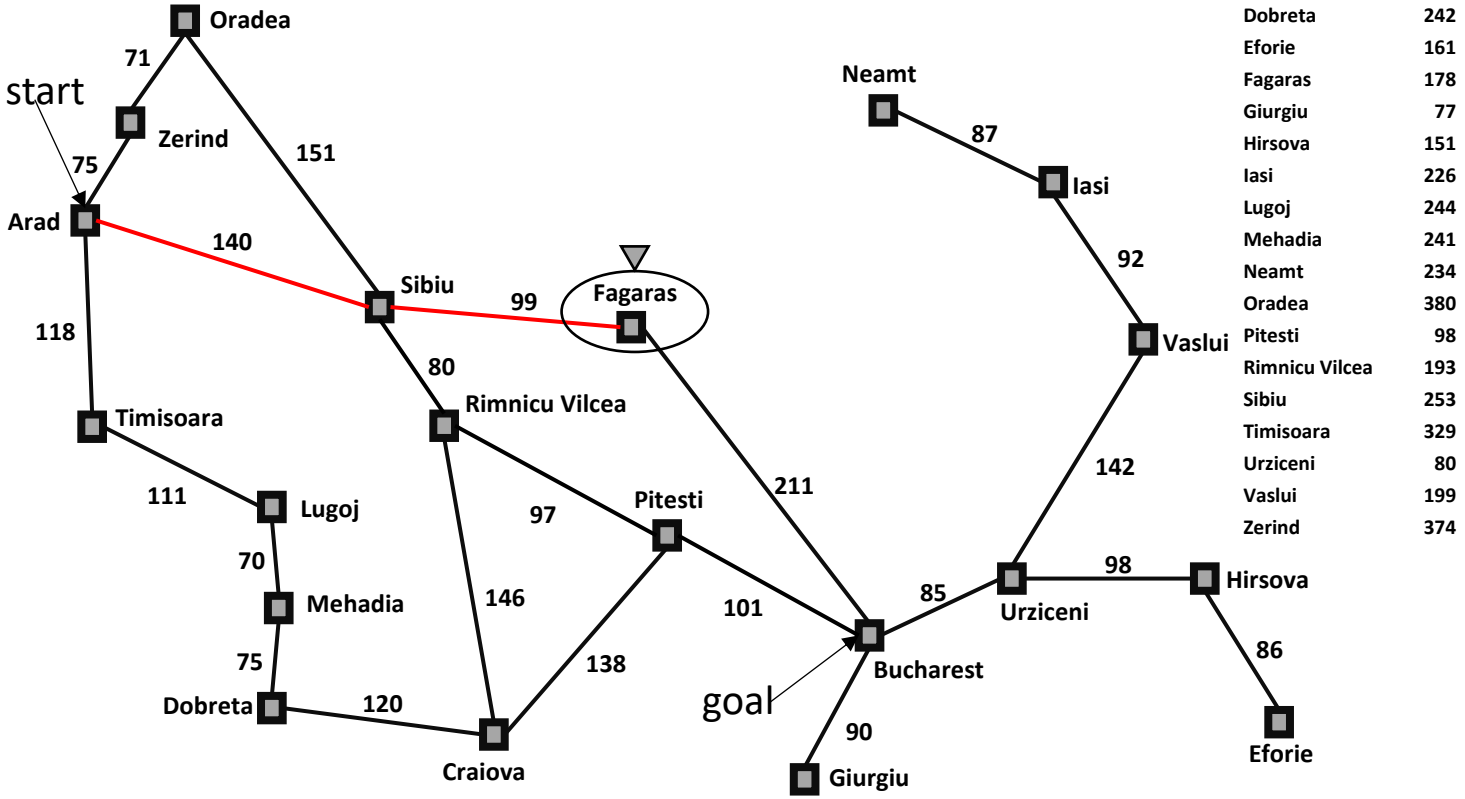
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



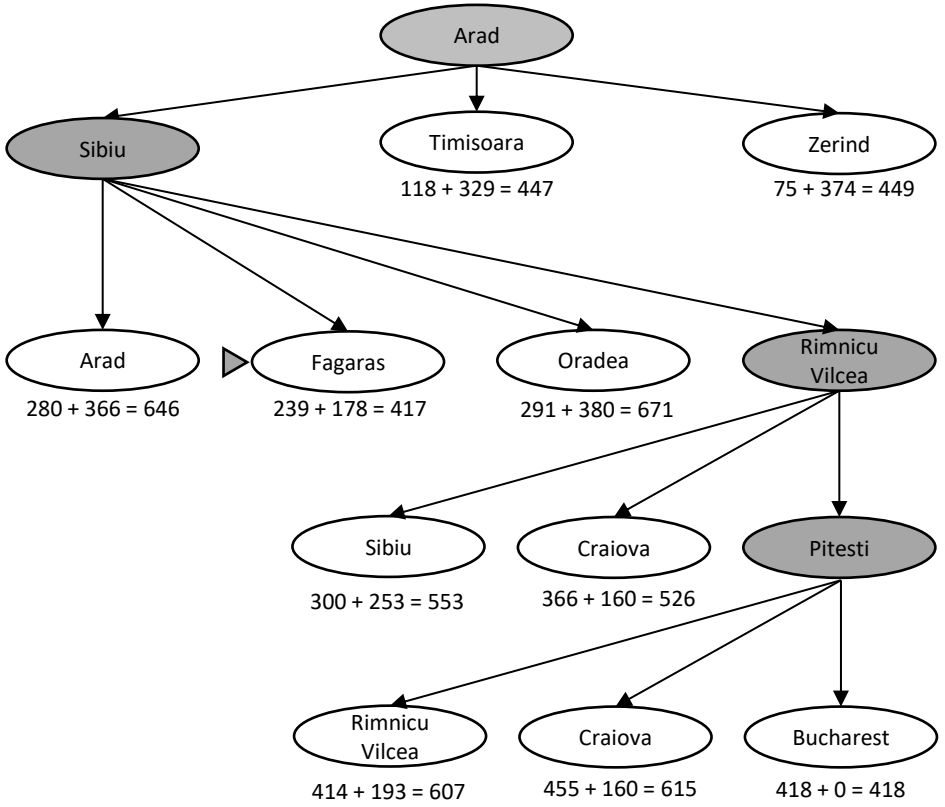
Node expansion in A*

- f(n) = g(n) + h(n)



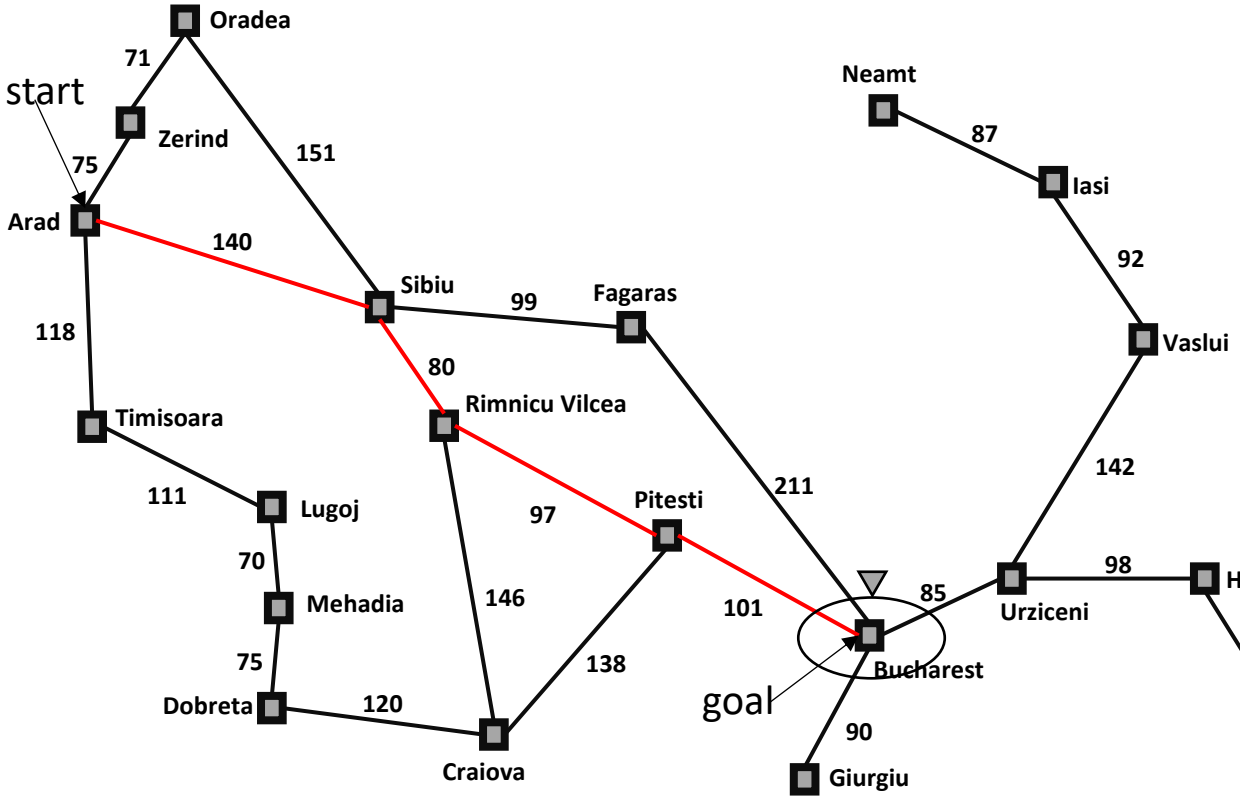
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



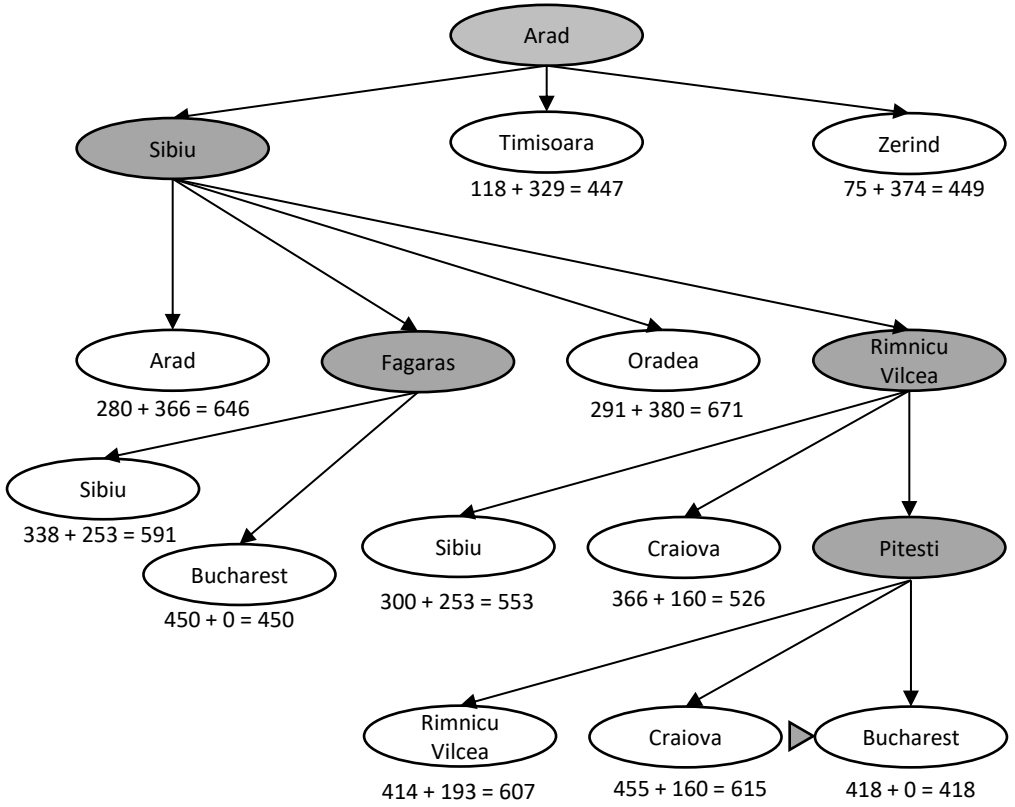
Node expansion in A*

- f(n) = g(n) + h(n)



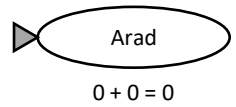
Straight-line distance to Bucharest

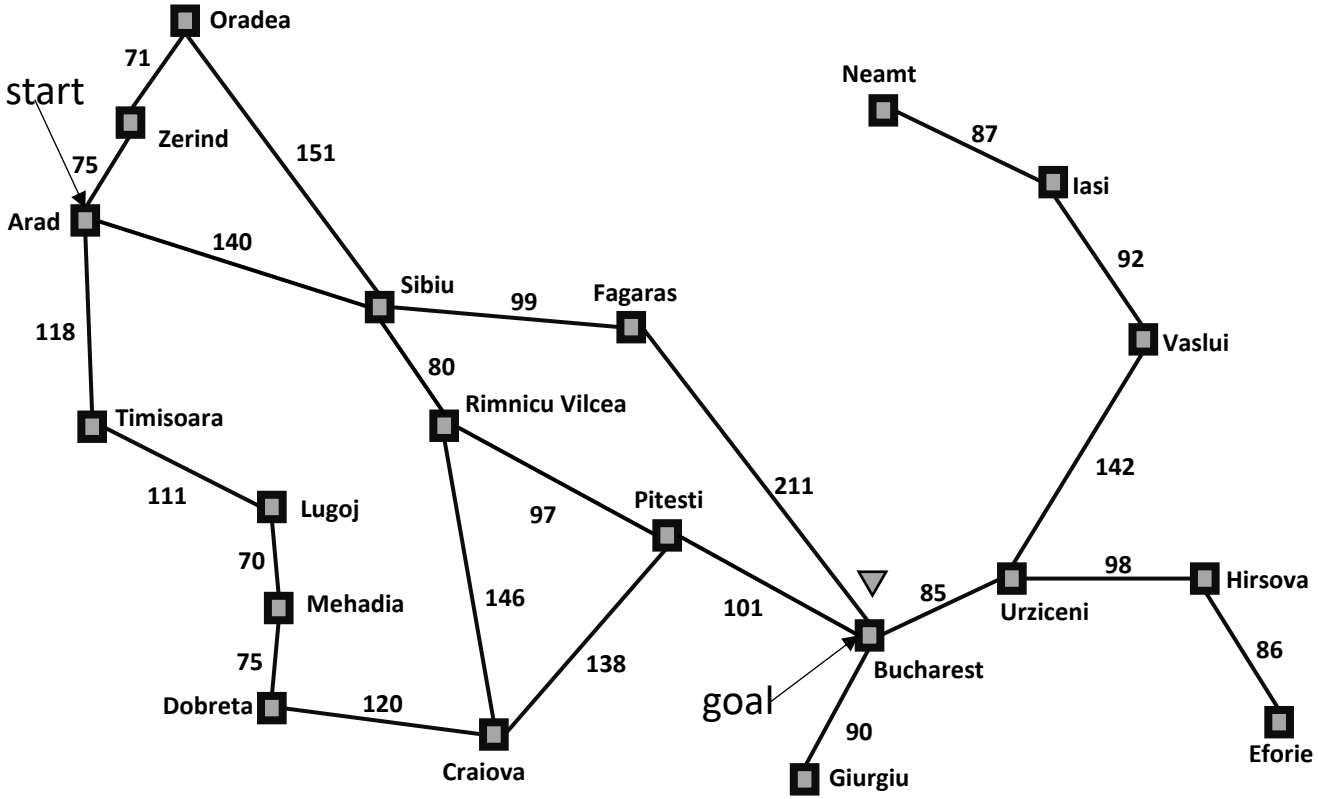
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Q

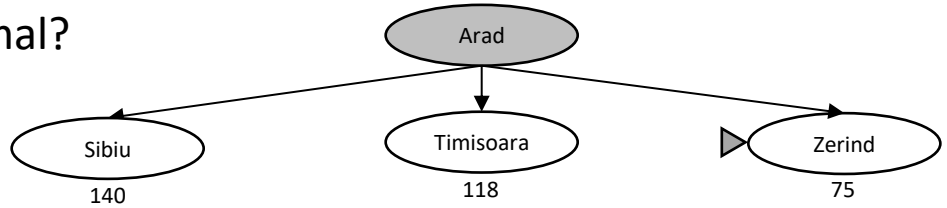
- What is A* search for the routing problem with h(n) = 0? Is it still optimal?

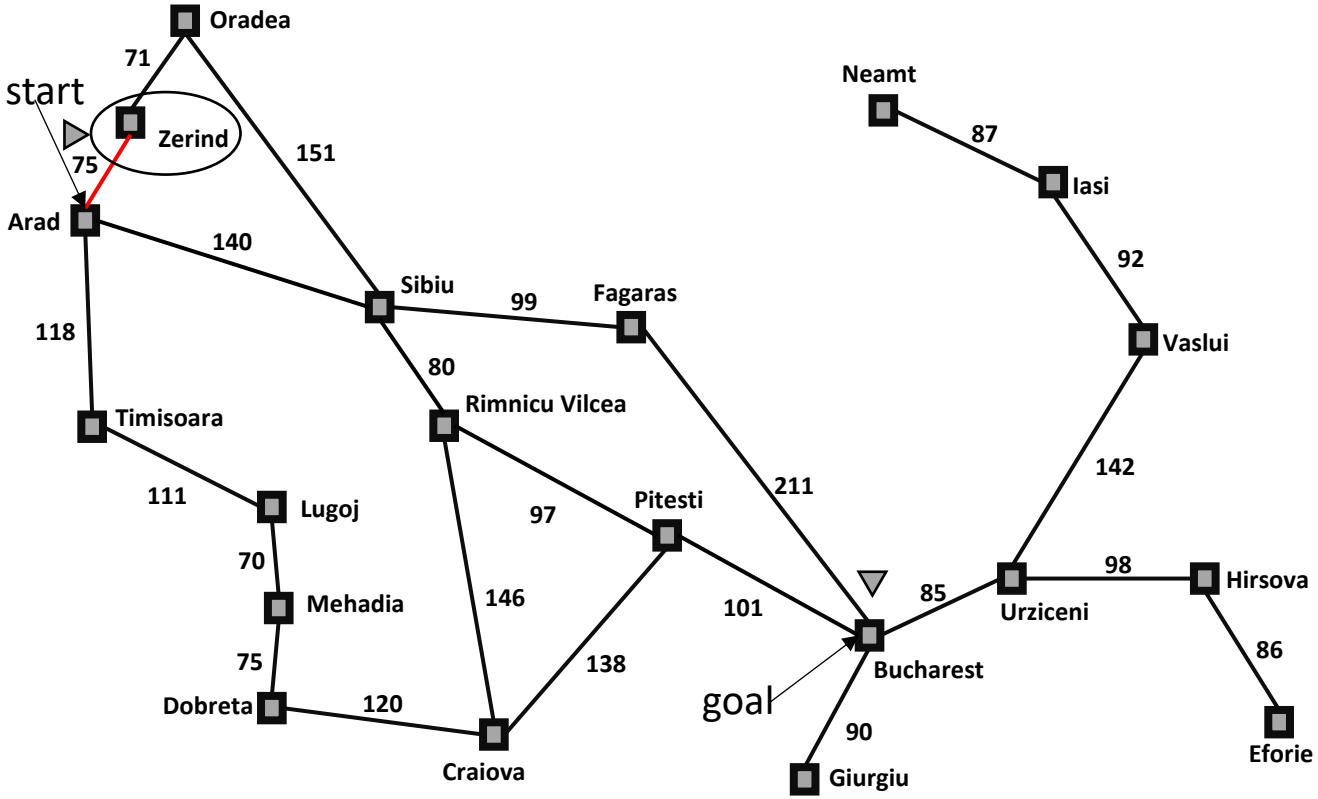




Q

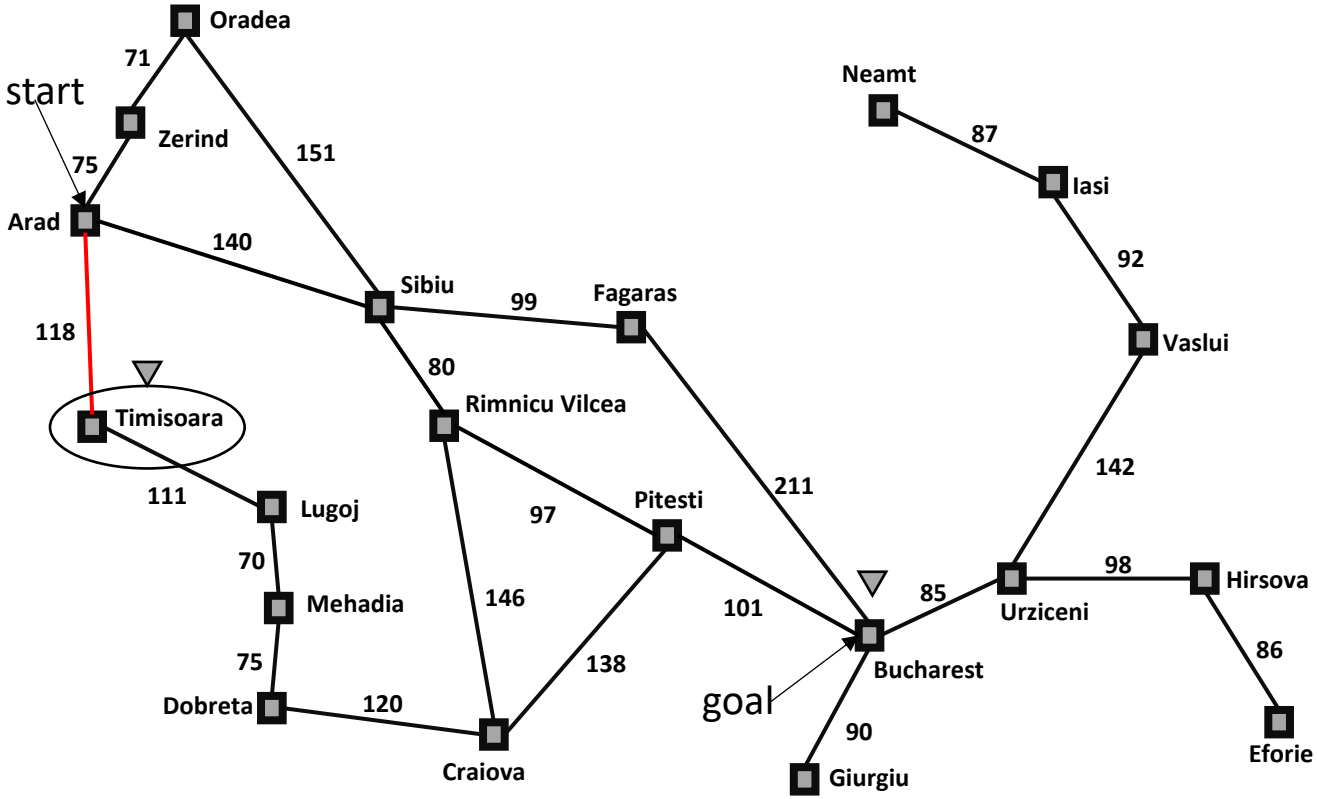
- What is A* search for the routing problem with h(n) = 0? Is it still optimal?
- f(n) = g(n)
  - This is Uniform-Cost Search (UCS) or Dijkstra

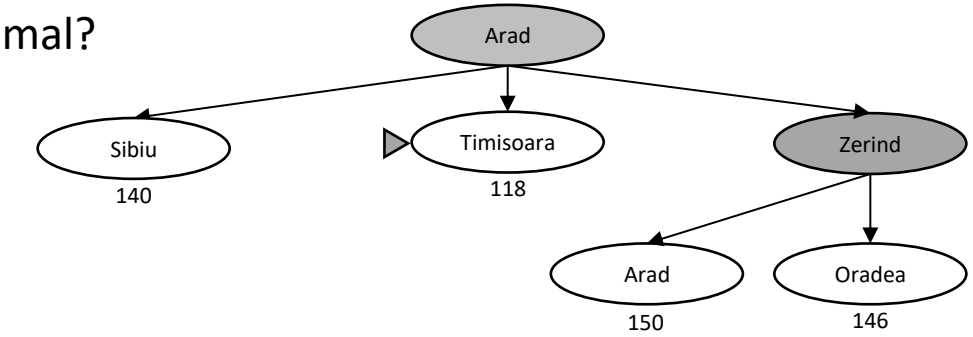




Q

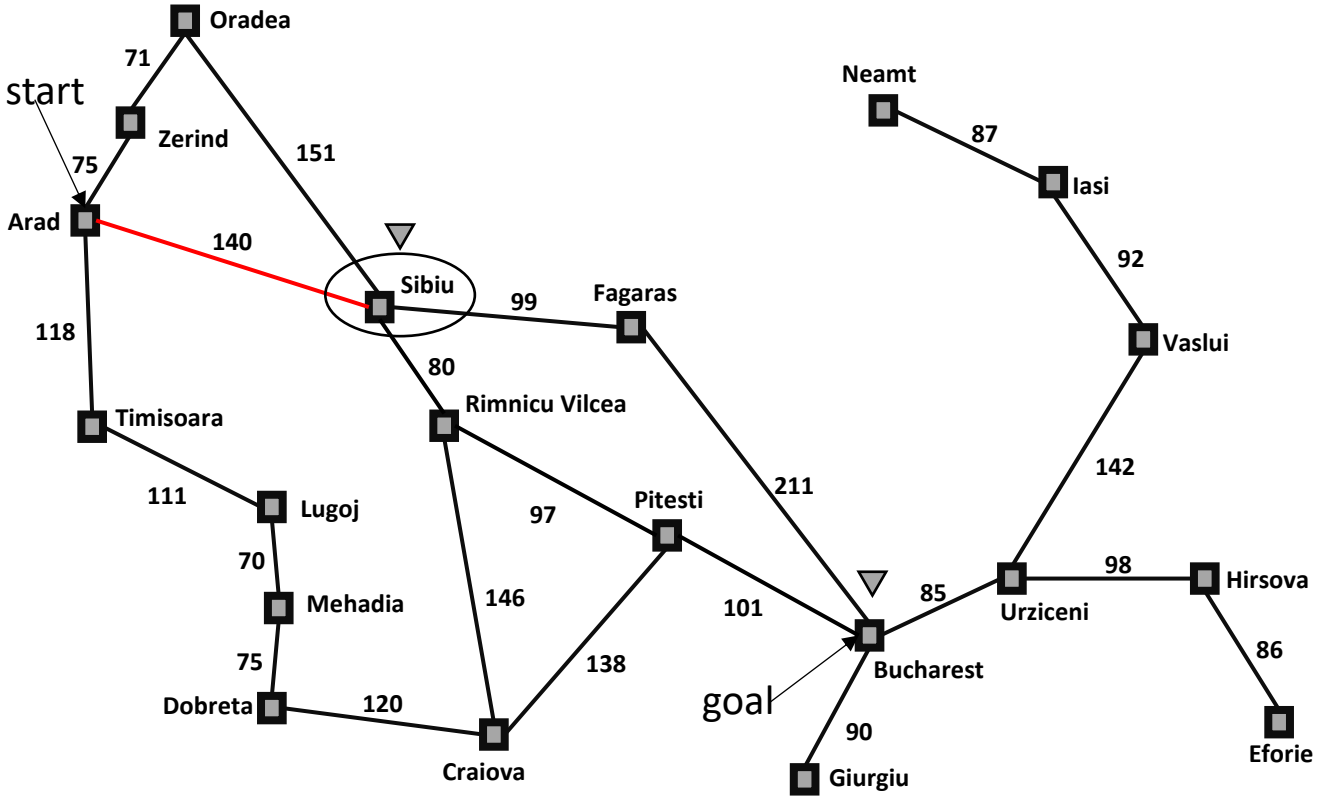
- What is A* search for the routing problem with h(n) = 0? Is it still optimal?
- f(n) = g(n)

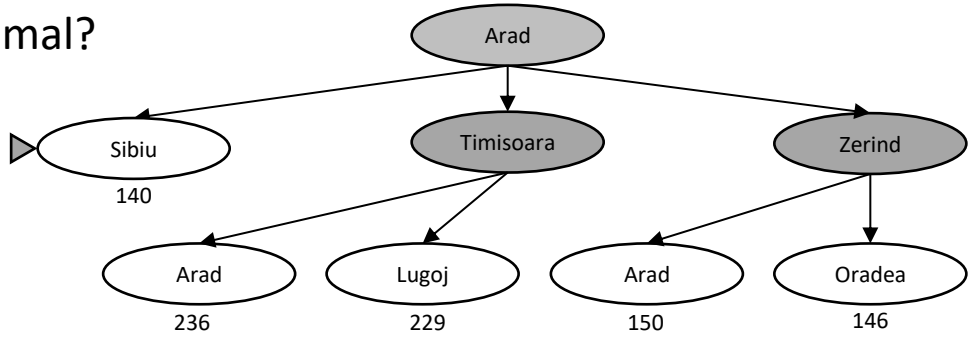




Q

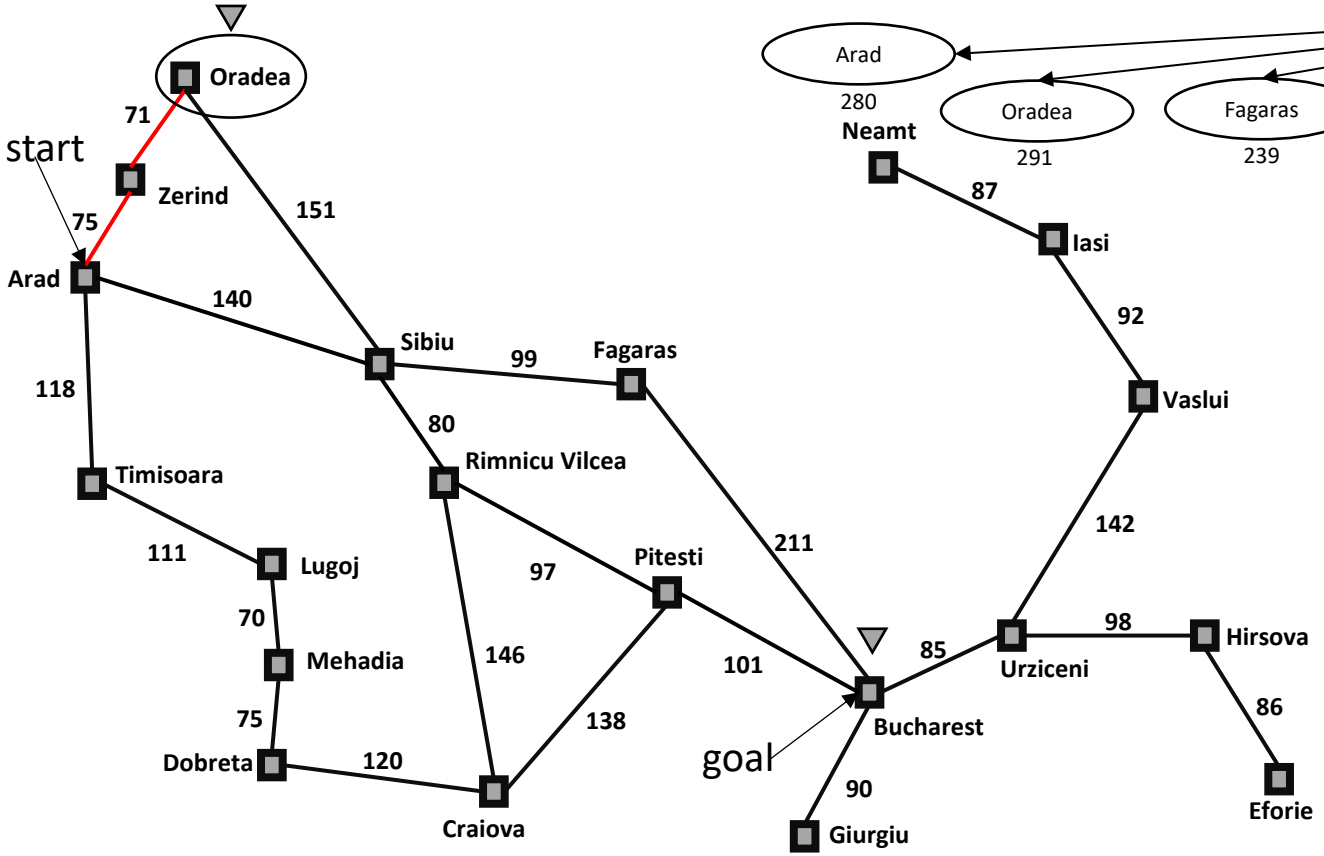
- What is A* search for the routing problem with h(n) = 0? Is it still optimal?
- f(n) = g(n)

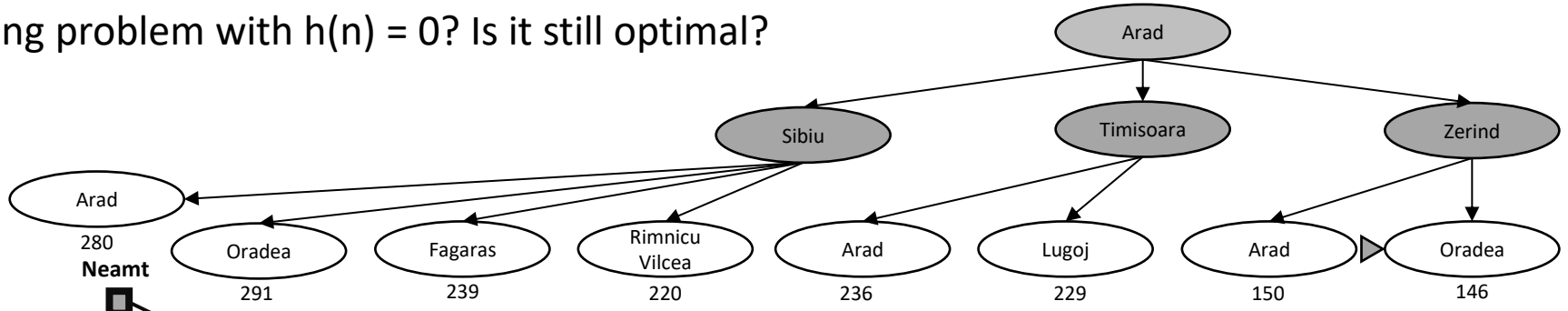




Q

- What is A* search for the routing problem with h(n) = 0? Is it still optimal?
- f(n) = g(n)





Properties of A*

- Complete: Yes
- Time: Exponential in length of solution
- Space: Keeps all nodes in memory
  - memory main problem
- Optimal: Yes (under certain conditions)

Variations of A*

- Memory can pose a problem for A*
- Variations
  - Iterative Deepening A* (IDA*)
  - Recursive best first search (RBFS)
  - Memory Bounded A* (MA*)
  - Simple MA (SMA*)

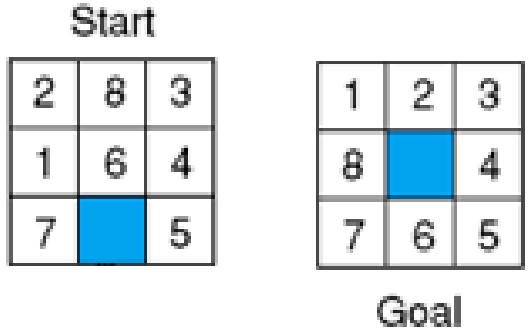
- See the book for the details of these methods

Design of heuristic function

- One of the keys to making your search efficient

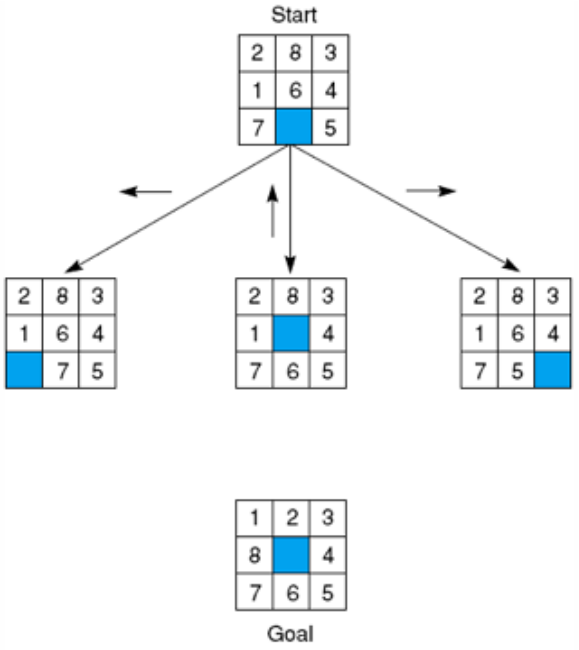
8-puzzle

- One of the first problems where heuristics was studied
- Any ideas for a heuristic?



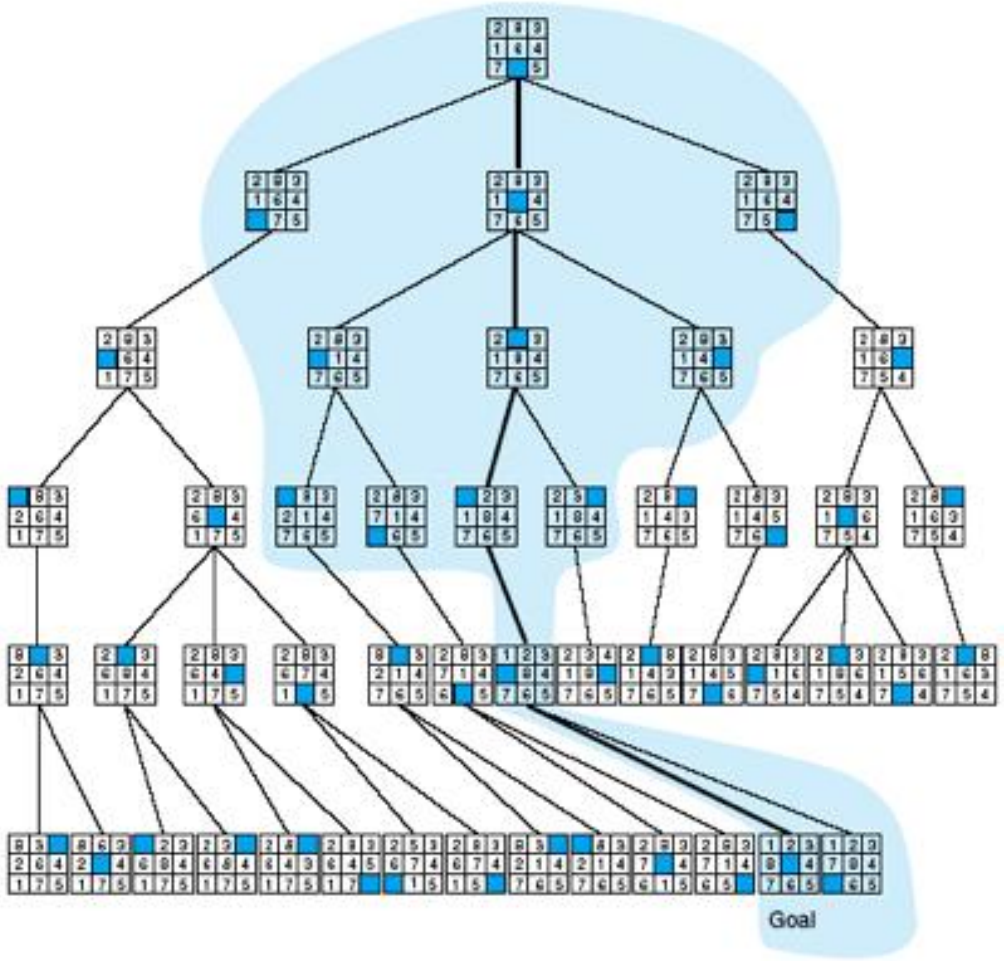
8-puzzle search tree example

- Two commonly used heuristics
  - $h_1(n)$: Number of misplaced tiles
  - $h_2(n)$: Sum of distances tiles are from goal position (measured using city-block distance)



8-puzzle with A*

- Notice how the search is “pulled” towards the goal
- Blue area marks explored part of search space



Dominating heuristics

- Which of the heuristics is better?
  - $h_1(n)$: Number of misplaced tiles
  - $h_2(n)$: Sum of distance tiles from goal position (measured using city-block distance)
- The one that needs to expand less nodes before finding an optimal solution
- If $h_2(n) \geq h_1(n)$ for all n, then h_2 dominates h_1
- Dominating heuristics is generally better (it leads to fewer node expansions)
  - Careful with not overestimating (you lose optimality)

Inventing heuristics

- Strategies to invent heuristics include
  - Look at relaxed problems
    - What are the relaxed problems for h_1 and h_2?
  - Analysis of sub-problems
    - What is a sub-problem of the 8-puzzle?
  - Use of pattern databases

Local search: when the path does not matter.



Local search methods / Optimization Problems

- In some cases, the path is irrelevant
- The goal configuration itself is the solution, not the path to it
  - train schedule, factory-floor layout, N-queens
- State space is the set of possible configuration
- Uses only a single current state, paths are not retained, and we want to try to keep improving the current state
- Not systematic, but only little memory is needed **and often find reasonable solutions**

Hill climbing search

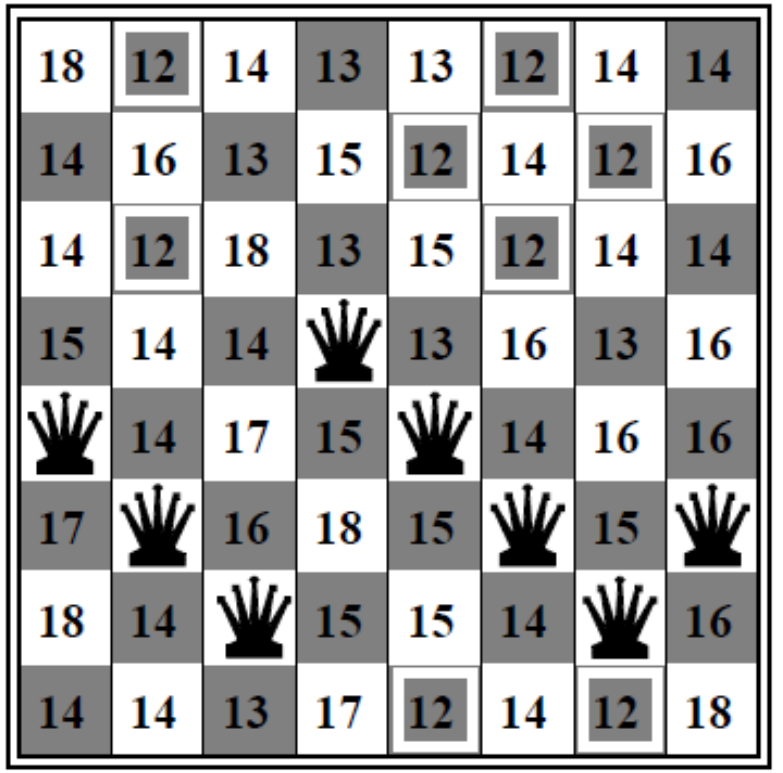
- Find the top of Mount Everest in a thick fog while suffering from amnesia
- Most basic (greedy) local search method
- Look at neighbors and move “uphill" in performance metric

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    inputs: problem, a problem
    local variables: current, a node
                     neighbor, a node

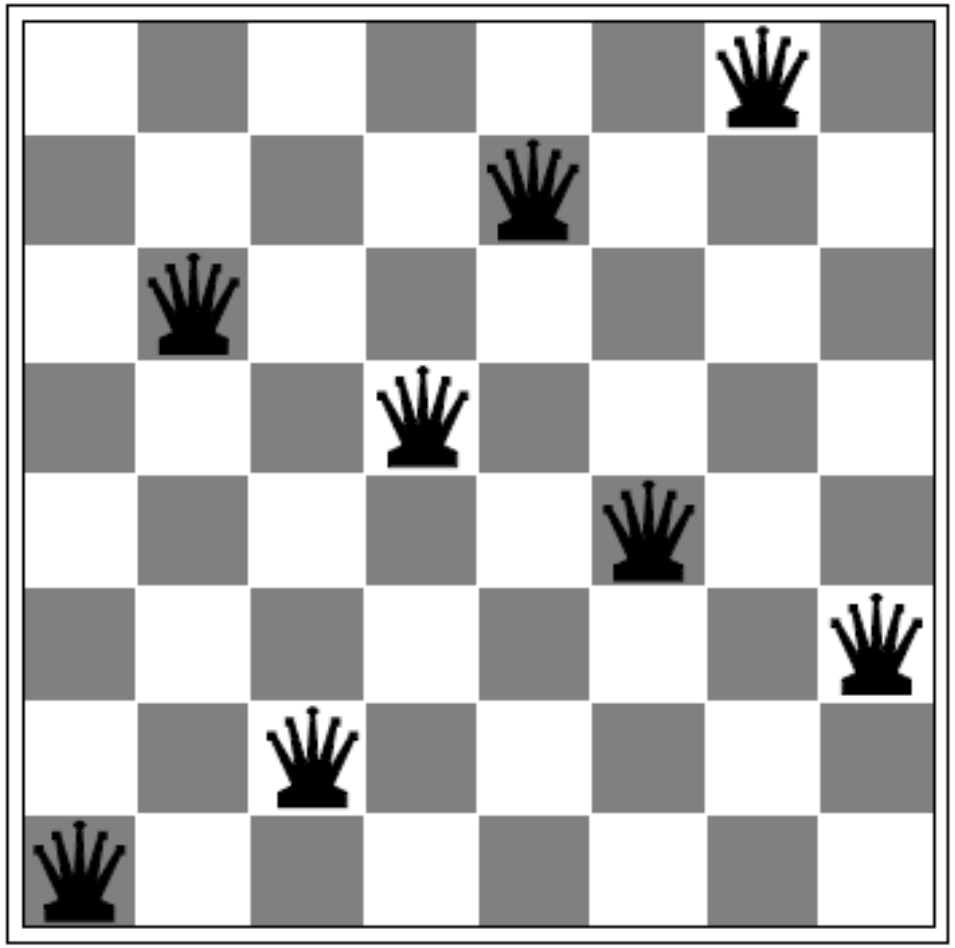
    current ← MAKE-NODE(INITIAL-STATE[problem])
    loop do
        neighbor ← a highest-valued successor of current
        if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
        current ← neighbor
```

Example: 8-queens

- h = Number of pairs of queens under attack
- h = 17 below
- The values on the board represent the cost to move a queen from the same column to that spot.



8-queens with local minima h=1



Hill climbing variations

- Stochastic hill climbing
  - Pick randomly among “improving successors"
- First-choice hill climbing
  - Pick randomly among all successors until finding an improving one
- Random restart hill climbing
  - Repeat with random initial states until goal is found

Simulated annealing

- Observation: Cannot move up in all landscapes in all steps!
- Make a “bad" move occasionally to escape.
- Reduce frequency/size over time

Summary of search methods

	Uninformed Path Search (BFS, DFS)	Informed Path Search (Greedy, A*)	Local Search (Hill Climbing)
Uses heuristics?	❌ No	✅ Yes	✅ Yes
Maintains paths?	✅ Frontier + paths	✅ Frontier + paths	❌ Usually current state only
What matters?	A path to a goal	A path to a goal	The final configuration
How next node/state chosen?	Structural rule: depth / frontier order	Evaluation such as h or $g + h$	Local improvement
Complete / Optimal?	Depends on algorithm	Greedy: not optimal; A*: yes under conditions	Generally, no guarantee
Memory	Algorithm-dependent	Usually maintains a frontier	Often very small

Questions

